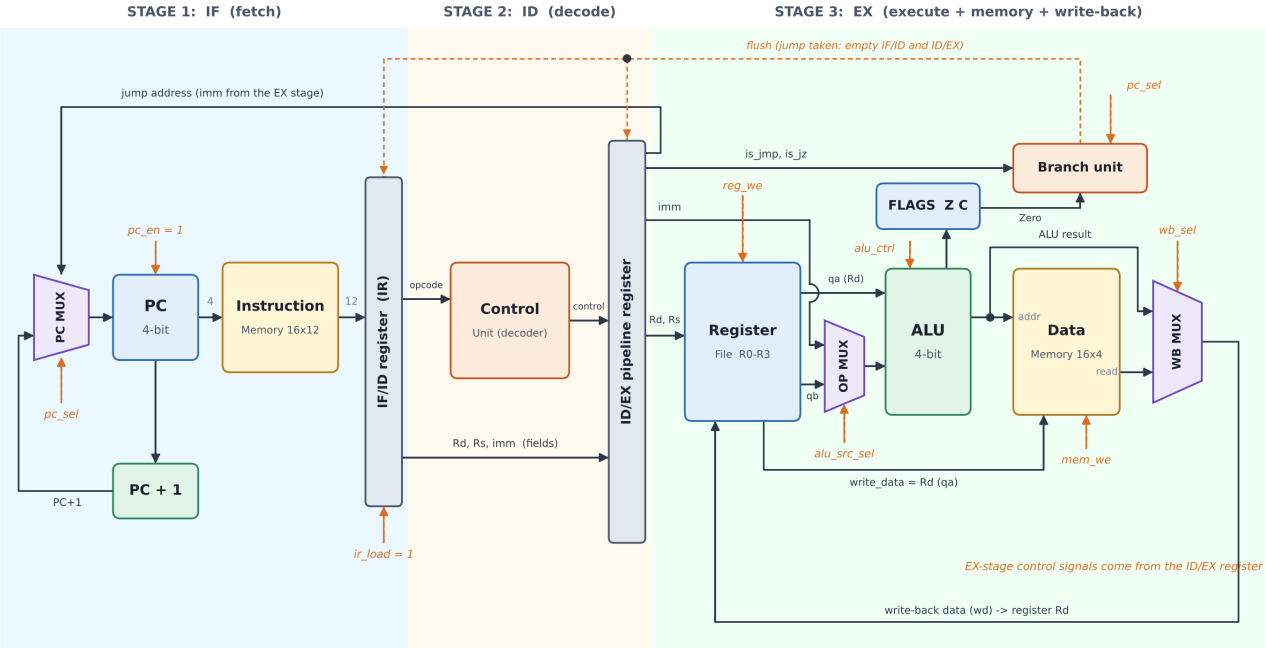
Design and Implementation of a

4-Bit RISC CPU Datapath

with a 3-Stage Pipeline, Using Verilog HDL



Project Report: Theory, Implementation and Verification

4-bit datapath · 12-bit instructions · 4 registers · IF / ID / EX pipeline · hardwired control · 88 / 88 simulation checks passed

Student / team	Guide	Institution	Duration
(add name)	(add name)	(add name)	2 weeks

October 2026

Contents

1. Introduction 6

1.1 Aim . . . 6

1.2 Objectives . . . 6

1.3 Key specifications . . . 7

2. Computer Architecture Fundamentals 8

2.1 Instruction Set Architecture (ISA) . . . 8

2.2 RISC versus CISC . . . 8

2.3 Von Neumann versus Harvard memory . . . 8

3. The Instruction Cycle and Pipelining 10

3.1 The steps of every instruction . . . 10

3.2 Three ways to use the clock . . . 10

3.3 What is a pipeline? . . . 10

3.4 Pipeline registers . . . 11

3.5 Pipeline hazards and how this CPU handles them . . . 12

3.6 Pipeline flowchart . . . 12

3.7 Relation to the single-cycle baseline in the project plan . . . 13

4. Digital Logic and Arithmetic Background 14

4.1 Combinational and sequential logic . . . 14

4.2 Combinational read, clocked write . . . 14

4.3 Multiplexers . . . 14

4.4 Unsigned and signed (two's complement) numbers . . . 14

4.5 The adder/subtractor . . . 15

4.6 Condition flags and the carry convention . . . 15

5. CPU Components 16

5.1 Program counter and next-address logic . . . 16

5.2 Instruction memory and the IF/ID register (IR) . . . 16

5.3 Control unit (decoder) and the ID/EX register . . . 17

5.4 Register file . . . 17

5.5 Operand multiplexer . . . 17

5.6 Arithmetic logic unit . . . 17

5.7 Flag register, data memory and write-back MUX . . . 18

5.8 Branch unit . . . 18

5.9 Hardwired control . . . 18

6. Instruction Set Architecture **19**

6.1 Instruction format . . . 19
6.2 Instruction set . . . 19
6.3 Addressing modes . . . 20
6.4 Encoding an instruction . . . 20

7. Control Unit Design **21**

7.1 Instruction decoder (ID stage) . . . 21
7.2 Control signals . . . 22
7.3 Decoder truth table . . . 22
7.4 Branch unit (EX stage) . . . 23

8. Datapath Schematic and Operation **24**

8.1 ADD Rd, Rs . . . 25
8.2 LDI Rd, imm . . . 25
8.3 LOAD Rd, addr . . . 26
8.4 STORE Rd, addr . . . 27
8.5 JZ addr (taken) . . . 27

9. Project Plan and Compliance **30**

9.1 Requirements traceability . . . 30
9.2 Module plan and interfaces . . . 30
9.3 Design decisions and differences from the plan . . . 31
9.4 Two-week schedule . . . 32
9.5 Risks and mitigation . . . 32

10. Project Structure and Coding Style **33**

10.1 Tools . . . 33
10.2 Coding style . . . 33
10.3 Module hierarchy and files . . . 33

11. Module Implementation **35**

11.1 ALU (alu.v) . . . 35
11.2 Register file (register_file.v) . . . 37
11.3 Program counter (program_counter.v) . . . 38
11.4 Instruction memory (instruction_memory.v) . . . 38
11.5 IF/ID pipeline register (instruction_register.v) . . . 39
11.6 ID/EX pipeline register (pipeline_register.v) . . . 40
11.7 Data memory (data_memory.v) . . . 41
11.8 Multiplexer (mux.v) . . . 42

11.9 Flag register (flag_register.v) . . . 42
11.10 Control unit (control_unit.v) . . . 43
11.11 Top level (cpu_top.v) . . . 46
12. Programs **50**
12.1 Writing a program by hand . . . 50
12.2 Demonstration program (program.mem, plan Appendix A) . . . 50
12.3 Test program: arithmetic and memory (prog_arith.mem) . . . 50
12.4 Test program: logic operations (prog_logic.mem) . . . 51
12.5 Test program: PC and jumps (prog_jump.mem) . . . 51
13. Verification **52**
13.1 Test plan coverage (project plan Section 8) . . . 52
13.2 Module-level testbenches . . . 53
13.3 CPU-level testbench . . . 57
14. Simulation Results **61**
14.1 Module-level results . . . 61
14.2 CPU-level results . . . 62
14.3 Pipeline charts . . . 63
14.4 Waveform: demonstration program . . . 65
14.5 Waveform: jumps . . . 65
14.6 Performance . . . 66
14.7 Acceptance criteria (project plan Section 8.1) . . . 67
15. Running the Project and Demonstration **68**
15.1 Simulation in Vivado . . . 68
15.2 Suggested final demonstration (plan Section 12) . . . 68
15.3 Optional FPGA demonstration . . . 68
16. Limitations, Extensions and Conclusion **69**
16.1 Limitations and possible extensions . . . 69
16.2 Conclusion . . . 69
Appendix: Quick Reference Card **70**

Part A

Theoretical Background
CPU concepts, pipelining, instruction set and control design

1. Introduction

A **Central Processing Unit (CPU)** reads instructions from memory and carries them out, one after another. Every CPU, from a small microcontroller to a desktop processor, is built from two halves:

- The **datapath** stores, selects and processes values: registers, the ALU, memories, multiplexers and the buses (groups of wires) that connect them.
- The **control path** (control unit) looks at each instruction and drives the select, enable and write-control signals of the datapath, so that the datapath does the right thing for that instruction.

This project designs, builds, connects and tests a small **4-bit RISC-style CPU** in Verilog HDL. "4-bit" means the registers, the ALU and the data memory all work with 4-bit numbers (0 to 15). The CPU uses a **3-stage pipeline**: while one instruction is being executed, the next one is being decoded and the one after that is being fetched. The design is small enough to understand completely, but it contains every important feature of a real processor: fetch, decode, execute, memory access, write-back, flags, jumps and pipelining. Each block is a separate Verilog module that is tested on its own before everything is connected.

1.1 Aim

To design and implement a functional 4-bit RISC CPU datapath using Verilog HDL and verify its operation through simulation and a structured testbench.

1.2 Objectives

1. Design a 4-bit arithmetic and logic unit (ALU) supporting arithmetic and bitwise operations.
2. Implement a four-register general-purpose register file.
3. Design the program counter, instruction memory, instruction register and datapath multiplexers.
4. Develop a hardwired control unit to decode instructions and generate datapath control signals.
5. Organise the CPU as a 3-stage pipeline (fetch, decode, execute) and handle jumps correctly.
6. Integrate the modules into a top-level CPU and execute short programs.
7. Verify functional behaviour using directed test cases and simulation waveforms.

1.3 Key specifications

Parameter	Specification
Processor type	Simple 4-bit RISC-style CPU, load/store architecture
Datapath width	4 bits (registers, ALU operands, data memory)
Register file	4 general-purpose registers R0-R3, 4 bits each; 2-bit register numbers
Instruction width	12 bits: opcode, two register fields and a 4-bit immediate
Instruction memory	16 words x 12 bits
Program counter	4 bits; addresses 16 instruction words
Data memory	16 words x 4 bits (for LOAD / STORE)
Pipeline	3 stages: IF (fetch), ID (decode), EX (execute + memory + write-back)
Speed	One instruction finishes every clock (after the pipeline fills); a taken jump costs 2 extra clocks
Control	Hardwired: combinational decoder + branch unit
Flags	Carry (arithmetic) and Zero (result = 0000)
Implementation	Verilog HDL, RTL simulation (Vivado Simulator / Icarus Verilog)
Project duration	2 weeks

Table 1. *Key specifications of the CPU*

2. Computer Architecture Fundamentals

2.1 Instruction Set Architecture (ISA)

The **Instruction Set Architecture** is the agreement between software and hardware. It says which instructions exist, how each one is written in binary, which registers a programmer can use, and how memory is addressed. In this project the 12-bit instruction format and the opcode table were fixed first (Chapter 6), and the hardware was then designed to carry them out. This is a teaching CPU, so it is not compatible with any commercial instruction set.

2.2 RISC versus CISC

Feature	RISC (Reduced Instruction Set)	CISC (Complex Instruction Set)
Instructions	Few, simple, regular	Many, some very complex
Instruction length	Fixed (here: always 12 bits)	Variable
Memory access	Only LOAD and STORE touch memory	Most instructions can use memory
Operands for arithmetic	Registers (or a small immediate)	Registers or memory
Decoding hardware	Simple, fast, hardwired	Complex, often microprogrammed
Pipelining	Easy, because every instruction looks alike	Harder
Examples	ARM, RISC-V, MIPS	x86

Table 2. *RISC versus CISC*

This CPU follows the RISC idea. The most important result is the **load/store architecture**: the ALU never works directly on memory. To add two numbers that are in memory, a program first loads them into registers, adds the registers, and then stores the answer back:

```
LOAD  R0, 3     ; R0 = data_memory[3]
LOAD  R1, 4     ; R1 = data_memory[4]
ADD   R0, R1    ; R0 = R0 + R1
STORE R0, 5     ; data_memory[5] = R0
```

2.3 Von Neumann versus Harvard memory

- **Von Neumann**: one memory holds both instructions and data, on one shared bus. The CPU cannot fetch an instruction and read or write data at the same moment.
- **Harvard**: instructions and data are kept in two separate memories, each with its own bus. A data store can never overwrite the program, and the two memories can even have different word sizes.

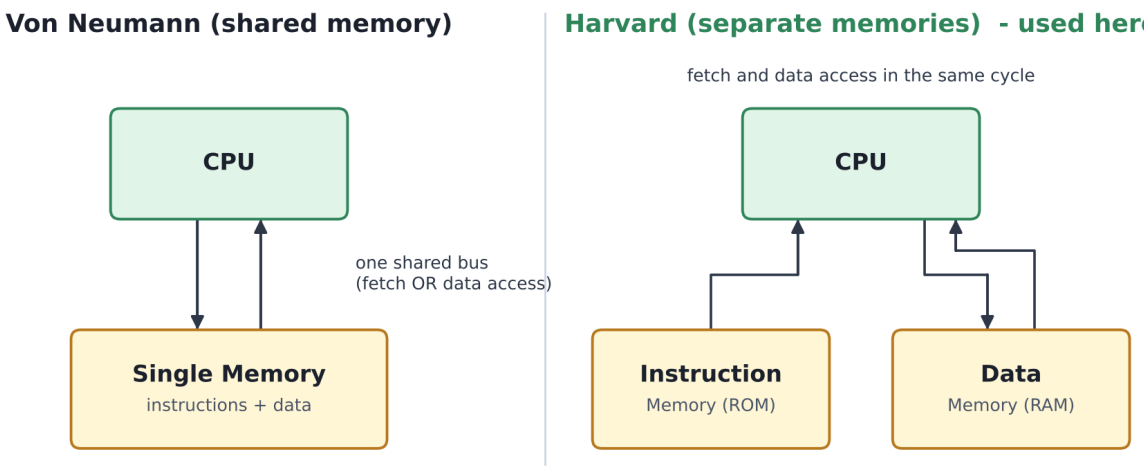


Figure 1. *Von Neumann (left) versus Harvard (right) memory. This CPU uses the Harvard model.*

The Harvard model is especially important for a pipeline: in the same clock cycle, the IF stage reads the instruction memory while the EX stage may be reading or writing the data memory. With one shared memory these two would clash. This CPU uses a 16 x 12-bit **instruction memory** and a separate 16 x 4-bit **data memory**.

3. The Instruction Cycle and Pipelining

3.1 The steps of every instruction

Every instruction goes through the same steps (project plan, Section 3.1):

1. **Fetch**: the PC addresses instruction memory and the instruction is loaded into the IR.
2. **Decode**: the control unit reads the opcode and makes the control signals; the register and immediate fields are separated.
3. **Operand read**: the register file gives the values of the source registers.
4. **Execute**: the ALU does the arithmetic or logic operation.
5. **Memory access**: LOAD or STORE uses the data memory.
6. **Write-back**: the result is written into the destination register.
7. **Next instruction**: the PC moves to PC + 1, or to a jump address.

In this CPU these steps are grouped into **three pipeline stages**: **IF** = step 1, **ID** = step 2, and **EX** = steps 3 to 7.

3.2 Three ways to use the clock

Design style	How it works	Clocks for 4 instructions	Comment
Single-cycle	All steps in one long clock cycle	4 (but each clock is long)	No separate fetch: the memory output drives everything directly
Multi-cycle (no pipeline)	One step-group per clock, one instruction at a time	12 (3 stages x 4)	Hardware is idle most of the time
3-stage pipeline (this CPU)	One stage per clock; all stages busy with different instructions	6	One instruction finishes every clock once the pipeline is full

Table 3. *Comparison of CPU timing organisations*

3.3 What is a pipeline?

A pipeline works like an assembly line in a factory. Instead of one worker building a whole product before starting the next one, the job is split into stations. Each station does its part and passes the product on. All stations work at the same time on different products, so a finished product comes off the line at every step.

In the CPU the three "stations" are IF, ID and EX. In each clock cycle:

- IF fetches instruction $n + 2$,
- ID decodes instruction $n + 1$,
- EX executes instruction n and writes its result.

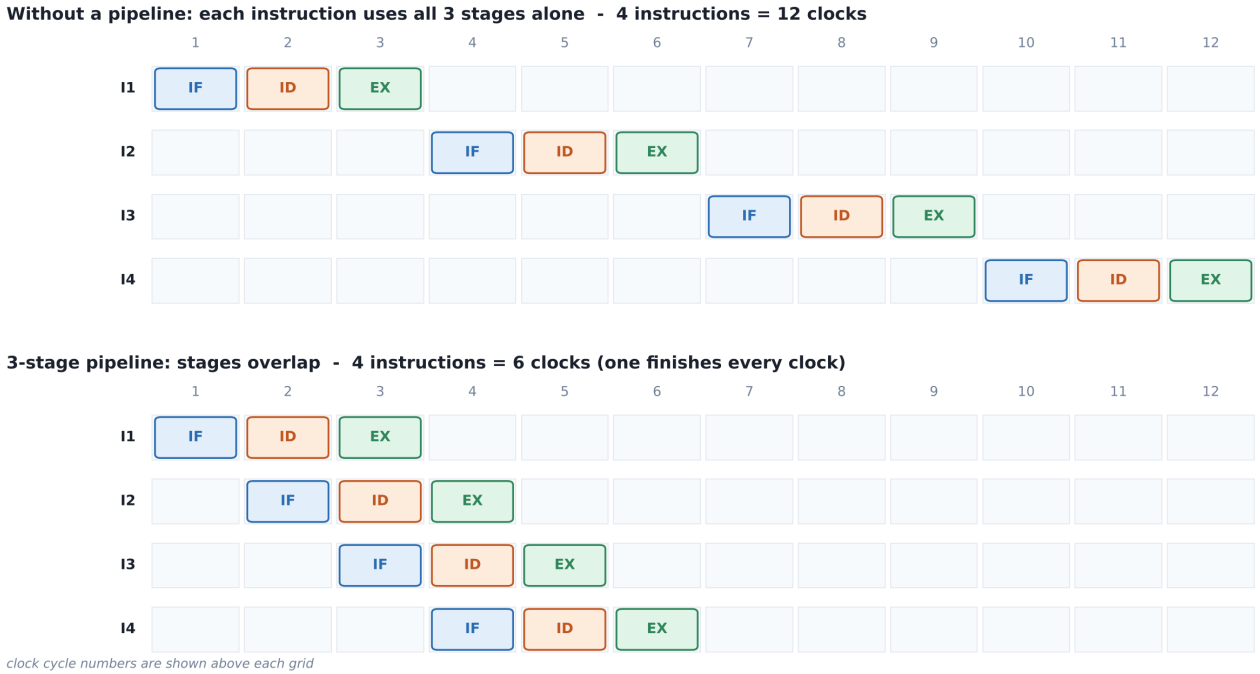


Figure 2. *Without a pipeline (top) each instruction uses the stages alone. With the 3-stage pipeline (bottom) the stages overlap and one instruction finishes every clock.*

Term	Meaning in this CPU
Latency	Time for ONE instruction to go through all stages: 3 clock cycles
Throughput	How many instructions finish per clock: 1 (when the pipeline is full)
Pipeline fill	The first instruction finishes in cycle 3; after that one finishes every cycle
N instructions (no jumps)	N + 2 clock cycles
Taken jump	2 extra clock cycles (2 instructions are flushed)

Table 4. *Pipeline words*

3.4 Pipeline registers

Each stage needs to keep its own instruction while the stage before it is already working on the next one. This is done with **pipeline registers** placed between the stages. At every clock edge, each pipeline register passes its contents one stage forward:

Pipeline register	Between	What it holds
IF/ID (the Instruction Register, IR)	IF and ID	the 12-bit instruction just fetched, its address, a valid bit
ID/EX	ID and EX	all control signals made by the decoder, plus Rd, Rs, imm, the address and a valid bit

Table 5. *The two pipeline registers*

3.5 Pipeline hazards and how this CPU handles them

A **hazard** is a situation where the overlap of instructions could give a wrong result. There are three kinds:

Hazard	What could go wrong	How this CPU avoids it
Structural	Two stages need the same hardware in the same clock	Separate instruction and data memories (Harvard); every block belongs to one stage
Data	An instruction reads a register before the previous instruction has written it	Registers are read in EX, one clock after the previous instruction wrote them, so the value is always new (same idea as the classic ARM7 3-stage pipeline)
Control	After a jump, the instructions already fetched are the wrong ones	The jump is decided in EX; the 2 wrong instructions in IF/ID and ID/EX are flushed (turned into NOPs)

Table 6. *Pipeline hazards*

Example of the data case: `LDI R1, 5` followed by `ADD R1, R2`. LDI writes R1 at the clock edge that ends its EX cycle. In the very next cycle ADD is in EX and reads R1, so it already sees 5. No waiting and no extra hardware is needed. The same is true for flags: SUB sets Zero at the end of its EX cycle and a following JZ tests it in the next cycle.

Example of the control case: when `JMP 4` (at address 1) is in EX, the instructions at addresses 2 and 3 have already entered ID and IF. The branch unit sets `pc_sel = 1` and `flush = 1`: at the clock edge the PC loads 4, and both pipeline registers are emptied. Addresses 2 and 3 never execute. The cost is 2 empty cycles ("bubbles").

3.6 Pipeline flowchart

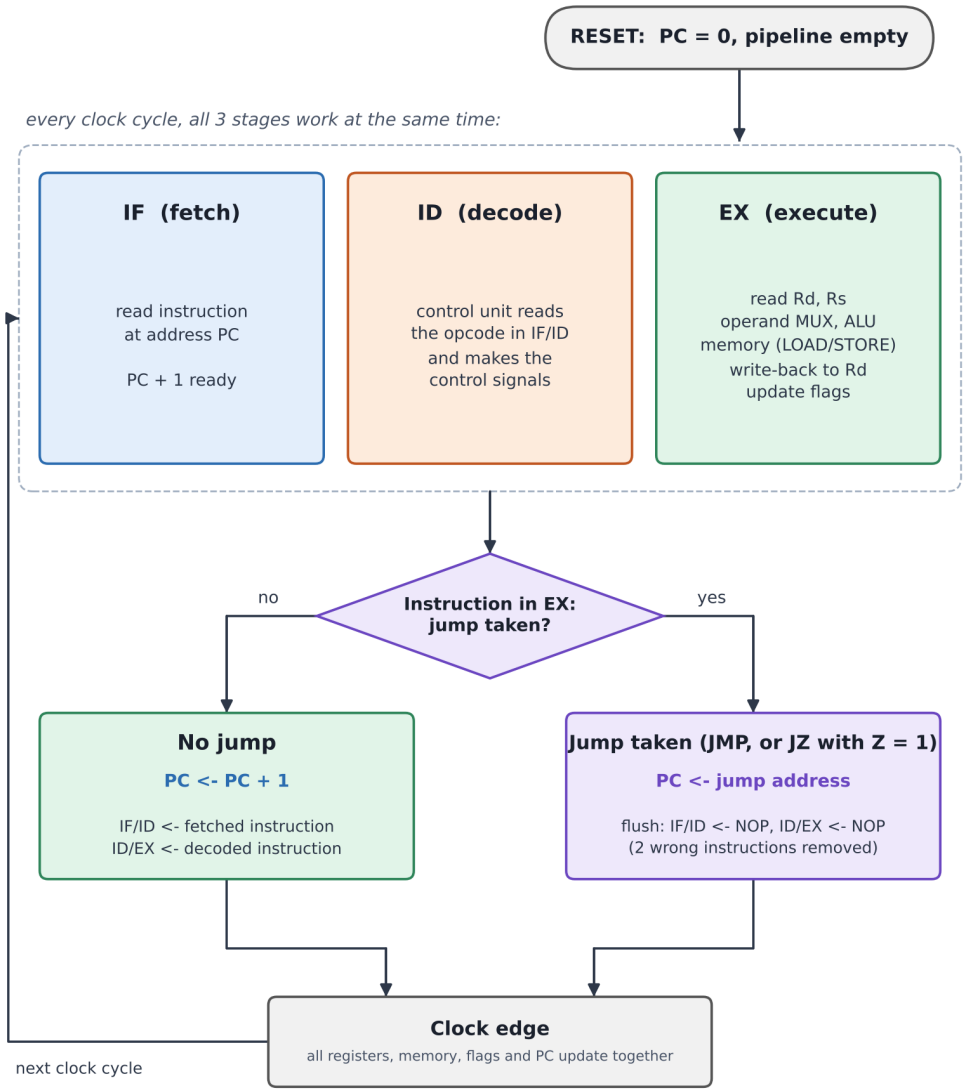


Figure 3. *What happens in every clock cycle of the pipelined CPU.*

3.7 Relation to the single-cycle baseline in the project plan

The project plan describes a single-cycle baseline in which "one instruction completes per active clock cycle", and it also asks for a clocked Instruction Register loaded during fetch. A pure single-cycle CPU cannot have a real fetch register, because the instruction would arrive only after the clock edge that loads it. The 3-stage pipeline satisfies both points: there is a real fetch stage with a clocked IR, and once the pipeline is full **one instruction still completes every clock cycle**. The only extra cost is 2 cycles at start-up and 2 cycles after each taken jump.

4. Digital Logic and Arithmetic Background

4.1 Combinational and sequential logic

- **Combinational logic** has no memory: its output depends only on its inputs right now. The ALU, the multiplexers, the decoder, the branch unit and the memory *read* paths are combinational. In Verilog: `assign` or `always @(*)`.
- **Sequential logic** remembers a value and changes only on a clock edge. The PC, the two pipeline registers, the register file, the flag register and the data-memory *write* are sequential. In Verilog: `always @(posedge clk)` with <=.

A pipelined CPU is a chain of sequential blocks (PC, IF/ID, ID/EX, registers) with combinational logic between them. At every rising clock edge, all of them capture their new values at the same moment.

4.2 Combinational read, clocked write

The register file and data memory are **read at any time** (the output follows the address immediately) and **written only on the rising clock edge**. So `ADD R1, R2` can read R1, compute R1 + R2 and write the answer back into R1 in the same EX cycle: during the cycle the old value is used, and the new value appears only after the edge.

4.3 Multiplexers

A **multiplexer (MUX)** chooses one of its inputs using a select signal. This CPU uses three 2-input, 4-bit multiplexers:

MUX	Input 0	Input 1	Select signal
Operand MUX	Register Rs value (qb)	Immediate imm[3:0]	alu_src_sel
Write-back MUX	ALU result	Data memory read data	wb_sel
PC MUX	PC + 1	Jump address (imm of the instruction in EX)	pc_sel

Table 7. *The three datapath multiplexers*

4.4 Unsigned and signed (two's complement) numbers

Bits	Unsigned	Signed	Bits	Unsigned	Signed
0000	0	0	1000	8	-8
0001	1	+1	1001	9	-7
0010	2	+2	1010	10	-6
0011	3	+3	1011	11	-5
0100	4	+4	1100	12	-4
0101	5	+5	1101	13	-3
0110	6	+6	1110	14	-2
0111	7	+7	1111	15	-1

Table 8. *4-bit values read as unsigned and as two's complement*

4.5 The adder/subtractor

Subtraction can be done with an adder, because A - B = A + (-B) = A + ~B + 1. In hardware, every bit of B goes through an XOR gate controlled by do_sub (XOR with 1 flips a bit), and do_sub is also the carry-in that gives the "+1".

```
wire [3:0] bx = b ^ {4{do_sub}};     // flip B when subtracting
wire [4:0] s  = a + bx + do_sub;     // 5 bits: s[4] is the carry
```

In the project code (Section 11.1) the same thing is written in an easier way (`result = a - b`), and the synthesis tool builds this adder hardware automatically.

4.6 Condition flags and the carry convention

Flag	Meaning	Updated by
Zero	The 4-bit result is 0000	ADD, SUB, AND, OR, XOR, NOT
Carry	ADD: carry out of bit 3. SUB: no-borrow (1 when A >= B)	ADD, SUB only

Table 9. *Condition flags*

> **Carry convention used everywhere in this project** (ALU, flag register, testbenches and this report): for ADD, Carry = 1 means the answer did not fit in 4 bits; for SUB, **Carry = 1 means no borrow** (A is greater than or equal to B) and Carry = 0 means a borrow happened. This is the convention used by ARM processors.

Operation	Binary	Result	Zero	Carry	Meaning
5 + 3	0101 + 0011	1000	0	0	Fits in 4 bits
15 + 1	1111 + 0001	0000	1	1	Too big for 4 bits, result zero
5 - 3	0101 - 0011	0010	0	1	No borrow (5 >= 3)
3 - 5	0011 - 0101	1110	0	0	Borrow (3 < 5)
7 - 7	0111 - 0111	0000	1	1	Zero, no borrow

Table 10. *Worked flag examples*

5. CPU Components

Block	Stage	Inputs / connection	Role / output
Program Counter (PC)	IF	clock, reset, pc_en, next address	address of the instruction to fetch
PC + 1 and PC MUX	IF	PC, jump address, pc_sel	next address: PC + 1 or jump target
Instruction Memory	IF	PC address	12-bit instruction word
IF/ID register (IR)	IF → ID	instruction, address, flush	holds the instruction being decoded
Control Unit: decoder	ID	opcode	all control signals for the instruction
ID/EX register	ID → EX	control signals, Rd, Rs, imm, flush	holds the instruction being executed
Register File	EX	read addresses, write address / data, write enable	two 4-bit operands; stores results
Operand MUX	EX	Rs value / immediate	ALU operand B
ALU	EX	two 4-bit operands, alu_ctrl	4-bit result, Carry, Zero
Flag Register	EX	ALU flags, write enables	stored Zero and Carry
Data Memory	EX	address (from ALU), write data (Rd), mem_we	read data for LOAD; store for STORE
Write-back MUX	EX	ALU result / memory data	value written into Rd
Control Unit: branch unit	EX	is_jmp, is_jz, Zero	pc_sel and flush

Table 11. *CPU blocks and the stage each belongs to*

5.1 Program counter and next-address logic

The PC holds the address of the instruction being fetched. Because the pipeline fetches a new instruction every clock, the PC changes every clock: normally to PC + 1, or to the jump address when the branch unit says a jump is taken. With 4 bits a program can have at most 16 instructions; after address 15 the PC wraps to 0.

5.2 Instruction memory and the IF/ID register (IR)

The instruction memory is a table of sixteen 12-bit words, loaded from a text file. Its output goes into the **IF/ID pipeline register**, which is the CPU's Instruction Register. The IR also keeps the instruction's address and a *valid* bit (0 means the slot is empty, a "bubble").

5.3 Control unit (decoder) and the ID/EX register

In the ID stage the decoder turns the opcode into control signals. These signals, together with Rd, Rs and imm, are saved in the **ID/EX pipeline register** so that the EX stage can use them in the next clock. When a jump is taken, the ID/EX register is loaded with all-zero control signals, so the flushed instruction changes nothing.

5.4 Register file

Four 4-bit registers with **two read ports** (qa, qb, read at any time) and **one write port** (clocked). Port A reads Rd and port B reads Rs. The CPU uses a **two-address** format, Rd = Rd op Rs: Rd is both the first source and the destination, which leaves room for a 4-bit immediate in the 12-bit word.

5.5 Operand multiplexer

Chooses ALU input B: Rs for register-register instructions, or the 4-bit immediate for LDI, LOAD and STORE. Passing the immediate through the ALU lets constants and memory addresses use the same path as normal results.

5.6 Arithmetic logic unit

Purely combinational. A shared adder/subtractor does ADD and SUB, a logic unit does AND, OR, XOR and NOT, and a result MUX chooses the answer using `alu_ctrl`. A zero detector makes Zero. A **PASS** operation copies operand B to the output; it is used by LDI, MOV, LOAD and STORE.

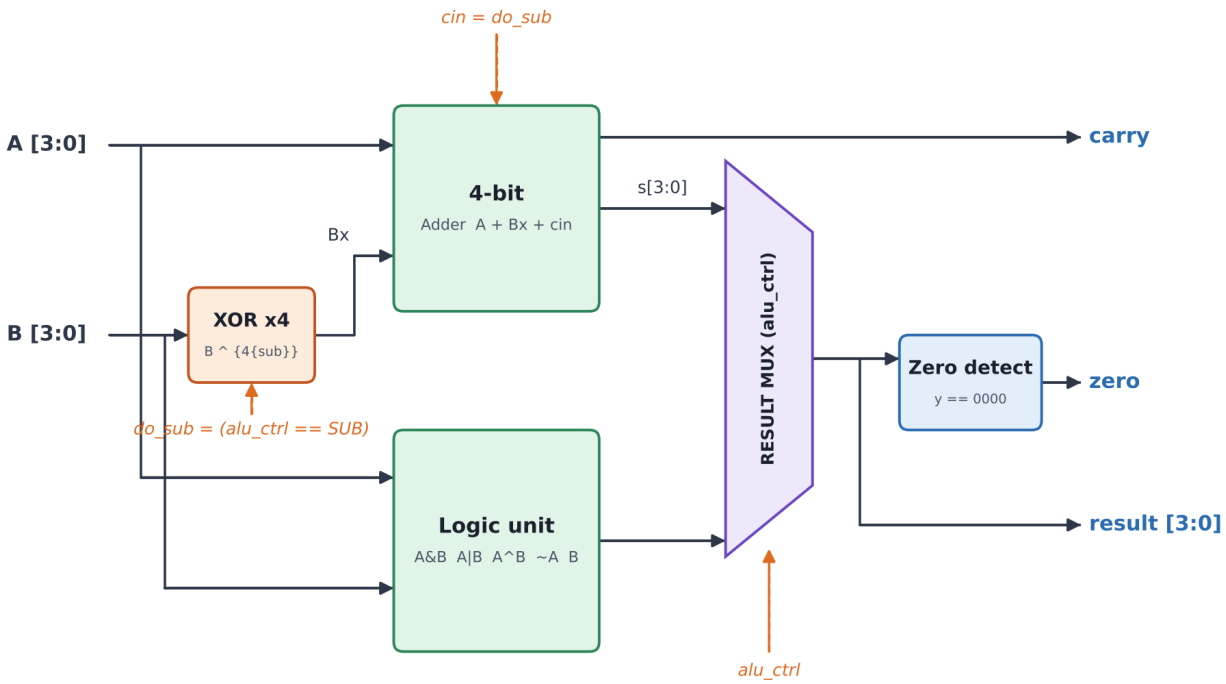


Figure 4. *Inside the ALU.*

alu_ctrl	Operation	result	Used by
0	ADD	a + b	ADD
1	SUB	a - b	SUB
2	AND	a & b	AND
3	OR	a | b	OR
4	XOR	a ^ b	XOR
5	NOT	~a	NOT
6	PASS	b	LDI, MOV, LOAD, STORE

Table 12. *ALU control codes*

5.7 Flag register, data memory and write-back MUX

The flag register stores Zero and Carry; they have separate write enables because logic instructions update Zero only. The 16 x 4-bit data memory takes its address from the ALU result, its write data from Rd, and is written only when `mem_we = 1`. The write-back MUX then chooses whether Rd receives the ALU result or the memory data.

5.8 Branch unit

The branch unit looks at the instruction in EX. It sets `pc_sel = 1` for JMP, or for JZ when the stored Zero flag is 1. The same signal is used as `flush`, which empties the two pipeline registers.

5.9 Hardwired control

A control unit can be **hardwired** (signals made directly by logic gates) or **microprogrammed** (signals stored as microinstructions in a small memory). Hardwired control is smaller and faster and suits a small RISC CPU, as the project plan asks.

6. Instruction Set Architecture

6.1 Instruction format

Every instruction is exactly 12 bits long and uses the same layout, so the decoder always finds the same field in the same place. An 8-bit word would be too small to hold an opcode, two register numbers and an immediate.

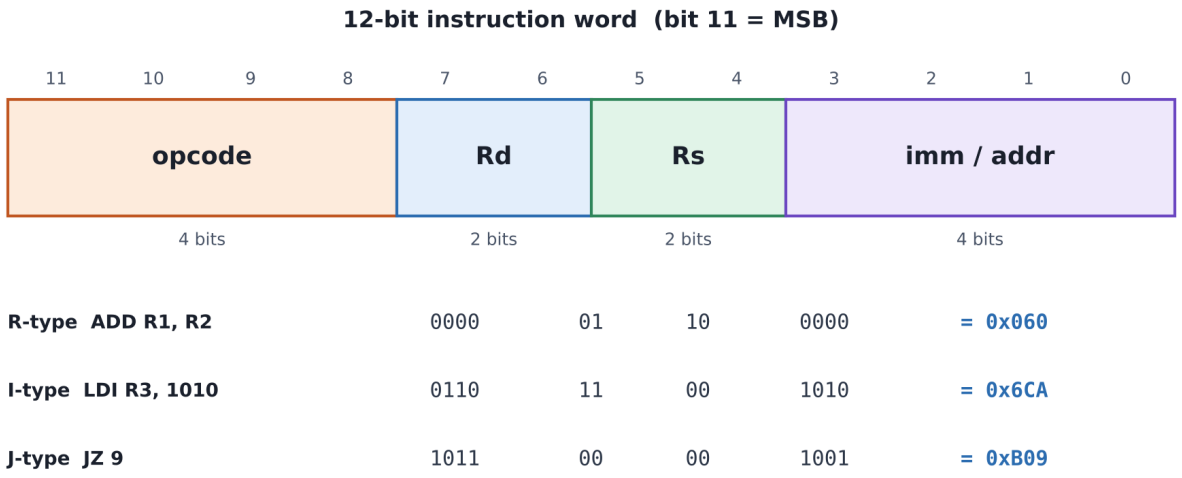


Figure 5. *12-bit instruction format with three encoding examples.*

Bits	Field	Width	Meaning
[11:8]	opcode	4	Operation (16 codes; 12 used, 4 reserved)
[7:6]	Rd	2	Destination register and first source (R0-R3)
[5:4]	Rs	2	Second source register (R0-R3)
[3:0]	imm / addr	4	Immediate number, data-memory address or jump address

Table 13. *Instruction fields*

6.2 Instruction set

Opcode	Mnemonic	Operation / behaviour	Flags
0000	ADD Rd, Rs	Rd ← Rd + Rs	Zero, Carry
0001	SUB Rd, Rs	Rd ← Rd - Rs (Carry = no borrow)	Zero, Carry
0010	AND Rd, Rs	Rd ← Rd AND Rs	Zero
0011	OR Rd, Rs	Rd ← Rd OR Rs	Zero
0100	XOR Rd, Rs	Rd ← Rd XOR Rs	Zero
0101	NOT Rd	Rd ← bitwise NOT Rd	Zero
0110	LDI Rd, imm	Rd ← immediate[3:0]	-
0111	MOV Rd, Rs	Rd ← Rs	-
1000	LOAD Rd, addr	Rd ← data_memory[addr]	-
1001	STORE Rd, addr	data_memory[addr] ← Rd	-
1010	JMP addr	PC ← addr	-
1011	JZ addr	If Zero = 1, PC ← addr; otherwise continue	-
1100-1111	(reserved)	No operation; free for future instructions	-

Table 14. *Instruction set (as in the project plan)*

6.3 Addressing modes

Mode	Where the operand comes from	Instructions
Register	A register named in the instruction	ADD, SUB, AND, OR, XOR, NOT, MOV
Immediate	A number written inside the instruction	LDI
Direct	A data-memory address inside the instruction	LOAD, STORE
Absolute jump	The jump address inside the instruction	JMP, JZ

Table 15. *Addressing modes*

6.4 Encoding an instruction

ADD R1, R2: opcode 0000, Rd = R1 = 01, Rs = R2 = 10, imm = 0000 → 0000 0110 0000 = **0x060**. LDI R3, 1010: 0110 11 00 1010 → **0x6CA**. STORE R1, 1: 1001 01 00 0001 → **0x941**. JZ 9: 1011 00 00 1001 → **0xB09**.

7. Control Unit Design

The control unit has two parts that work in different pipeline stages:

- the **instruction decoder** in the ID stage, which turns the opcode into control signals, and
- the **branch unit** in the EX stage, which decides whether a jump is taken and whether to flush.

7.1 Instruction decoder (ID stage)

The decoder is combinational logic written with **safe defaults**: every signal is first set to its "do nothing" value, and the case statement only turns on the signals an instruction needs. This gives a fixed, known output for every opcode, avoids unwanted latches, and makes the reserved opcodes act as NOP.

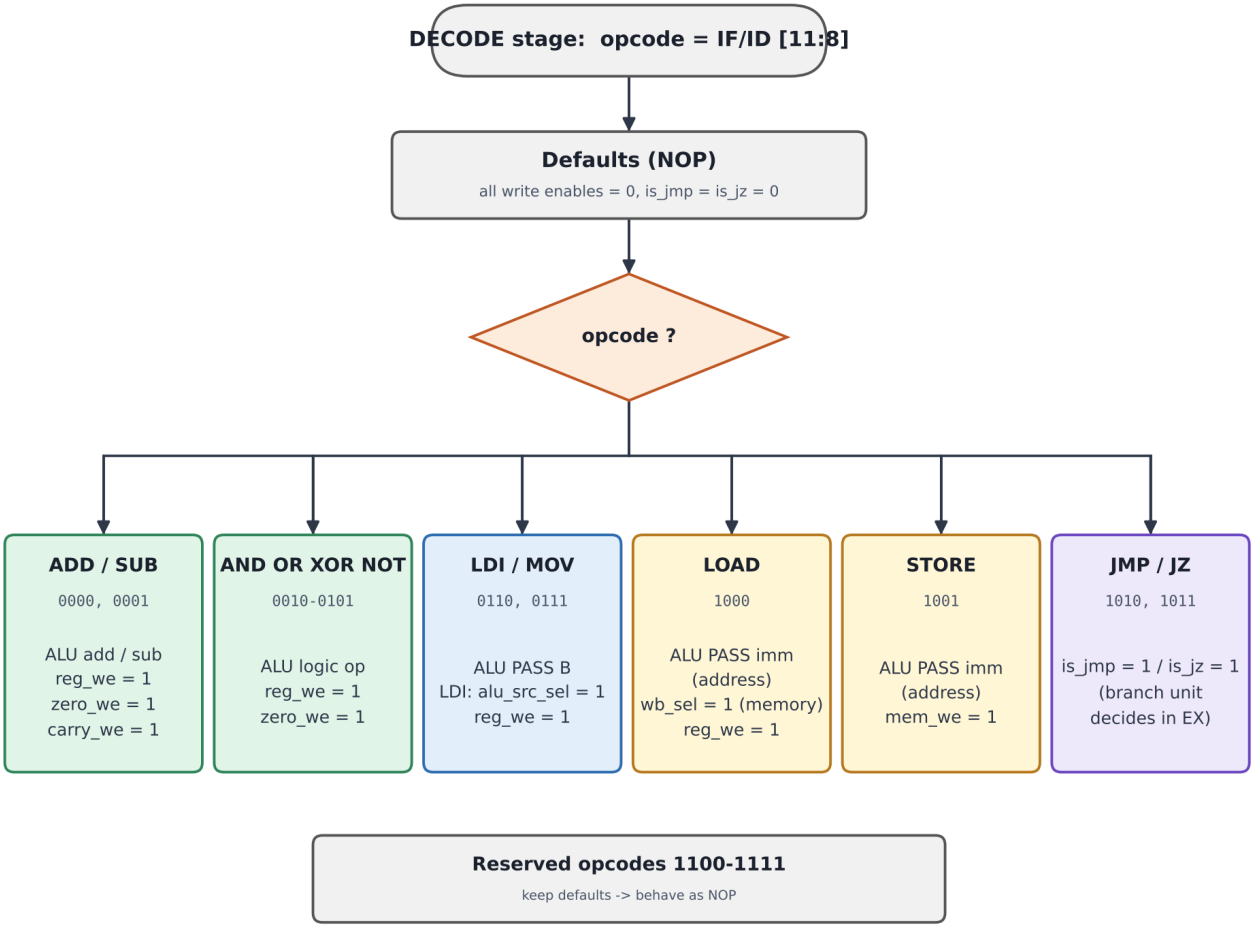


Figure 6. *Decoder flowchart: opcode groups and the control signals each group turns on.*

7.2 Control signals

Signal	Made in	Used in	Meaning
`alu_ctrl (3 bits)`	ID	EX	ALU operation
`alu_src_sel`	ID	EX	Operand MUX: 0 = Rs, 1 = immediate
`wb_sel`	ID	EX	Write-back MUX: 0 = ALU result, 1 = memory data
`reg_we`	ID	EX	Write the destination register Rd
`mem_we`	ID	EX	Write the data memory
`zero_we, carry_we`	ID	EX	Update the Zero / Carry flag
`is_jmp, is_jz`	ID	EX	Tell the branch unit what kind of jump this is
`pc_sel`	EX (branch unit)	IF	PC MUX: 0 = PC + 1, 1 = jump address
`flush`	EX (branch unit)	IF/ID, ID/EX	Empty both pipeline registers
`pc_en, ir_load`	fixed at 1	IF	Fetch a new instruction every clock

Table 16. *Control signals*

7.3 Decoder truth table

Instr	alu_ctrl	alu_src	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jmp	is_jz
ADD	ADD	0	0	1	0	1	1	0	0
SUB	SUB	0	0	1	0	1	1	0	0
AND	AND	0	0	1	0	1	0	0	0
OR	OR	0	0	1	0	1	0	0	0
XOR	XOR	0	0	1	0	1	0	0	0
NOT	NOT	0	0	1	0	1	0	0	0
LDI	PASS	1	0	1	0	0	0	0	0
MOV	PASS	0	0	1	0	0	0	0	0
LOAD	PASS	1	1	1	0	0	0	0	0
STORE	PASS	1	0	0	1	0	0	0	0
JMP	-	0	0	0	0	0	0	1	0
JZ	-	0	0	0	0	0	0	0	1
reserved	-	0	0	0	0	0	0	0	0

Table 17. *Decoder outputs for every opcode (checked by tb_control_unit)*

7.4 Branch unit (EX stage)

Instruction in EX	Zero flag	pc_sel	flush	What happens at the clock edge
not a jump	any	0	0	PC ← PC + 1; pipeline moves forward normally
JMP	any	1	1	PC ← jump address; IF/ID and ID/EX emptied
JZ	0	0	0	jump not taken; continue normally
JZ	1	1	1	PC ← jump address; IF/ID and ID/EX emptied

Table 18. *Branch unit truth table (checked by tb_control_unit)*

A conditional jump is just a multiplexer. For JZ, pc_sel is simply the stored Zero flag: if it is 1 the PC MUX picks the jump address, otherwise it picks PC + 1.

8. Datapath Schematic and Operation

The schematic below shows the whole CPU divided into its three pipeline stages. It shows every bus and control line asked for in the project plan: the 12-bit instruction bus, the 4-bit data and address buses, the register write enable, the three MUX select lines, the ALU control and the PC controls. Solid lines carry data; dashed orange arrows are control signals. A small hop shows two lines crossing without connecting; a dot shows a real connection.

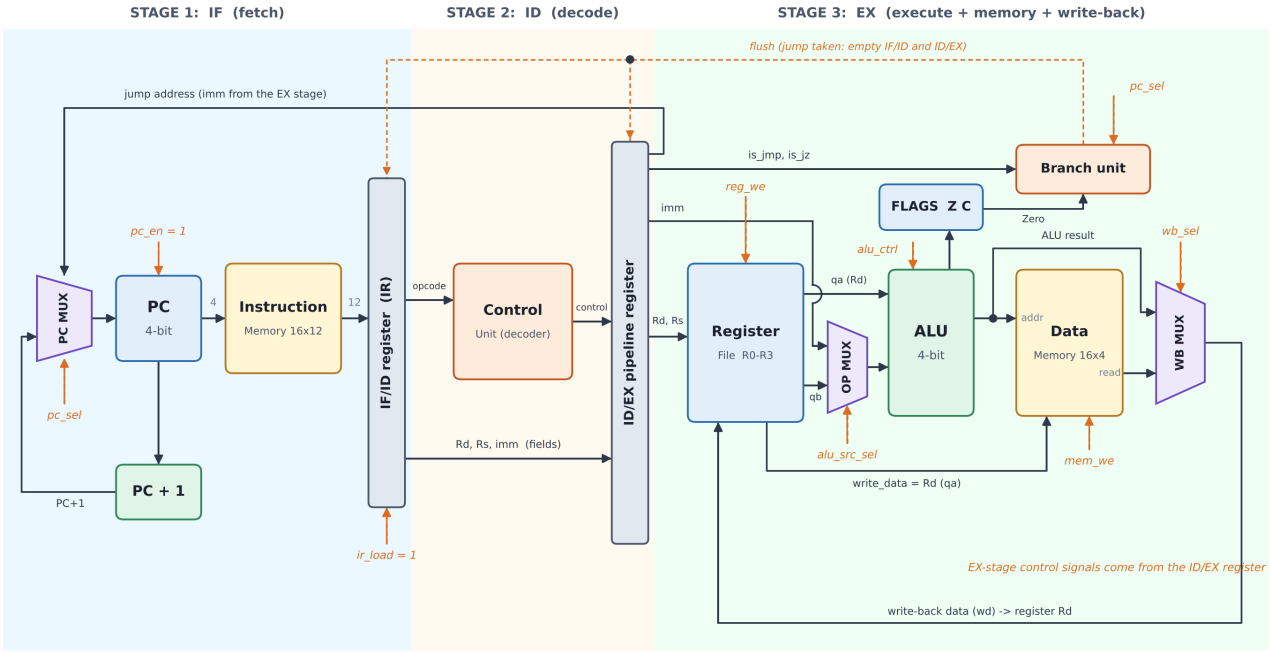


Figure 7. *Complete datapath of the pipelined 4-bit RISC CPU.*

Path	Route	Width
Instruction path	PC → instruction memory → IF/ID (IR)	4-bit address, 12-bit instruction
Control path	IF/ID opcode → decoder → ID/EX → MUX selects, ALU, register / memory write enables	4-bit opcode, control lines
Data path	Register file → operand MUX → ALU → write-back MUX → register file	4 bits
Memory path	ALU result → data-memory address; Rd → write data; read data → write-back MUX	4 bits
Jump path	ID/EX imm → PC MUX → PC; branch unit → pc_sel and flush	4 bits + control

Table 19. *Block-level paths (project plan Section 3.2)*

The next pages show, in red, the path each kind of instruction uses as it moves through the three stages.

8.1 ADD Rd, Rs

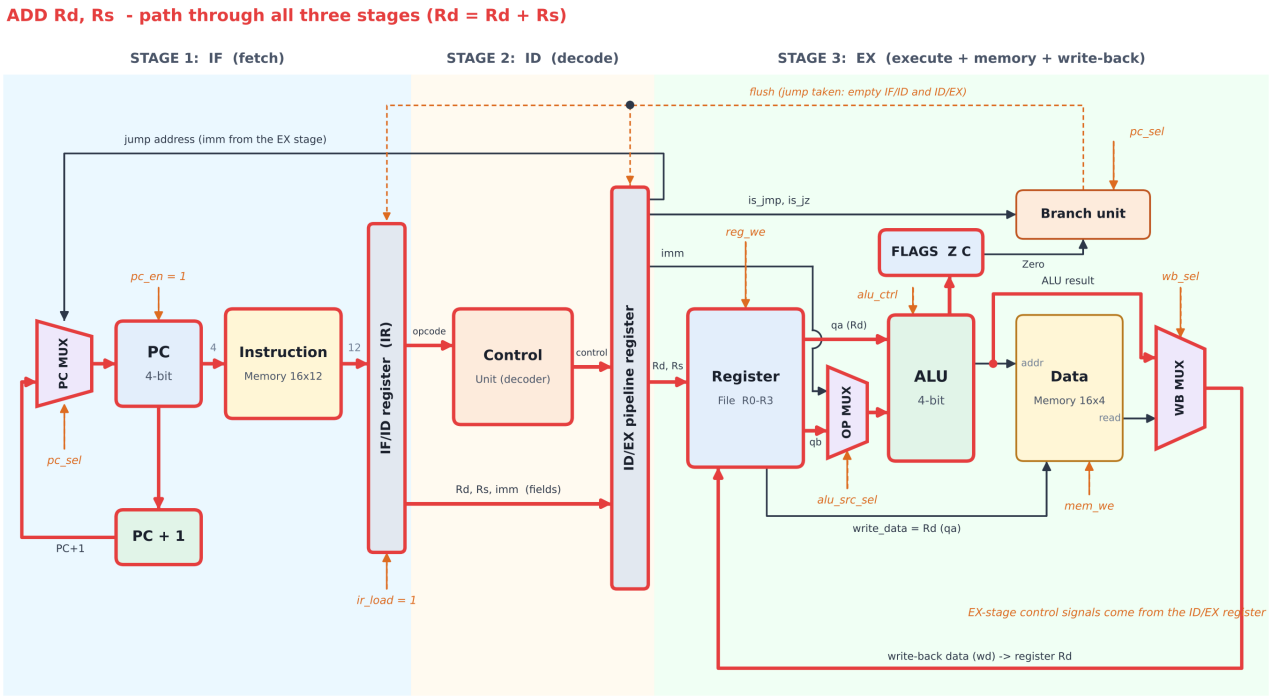


Figure 8. *Path of ADD Rd, Rs through the pipeline.*

1. **IF (clock 1):** the instruction is read from address PC and saved in IF/ID.
2. **ID (clock 2):** the decoder makes alu_ctrl = ADD, reg_we = 1, zero_we = carry_we = 1; these are saved in ID/EX.
3. **EX (clock 3):** Rd and Rs are read, the operand MUX passes Rs (alu_src_sel = 0), the ALU adds, wb_sel = 0.
4. At the end of clock 3: Rd ← sum, Zero and Carry ← flags.

8.2 LDI Rd, imm

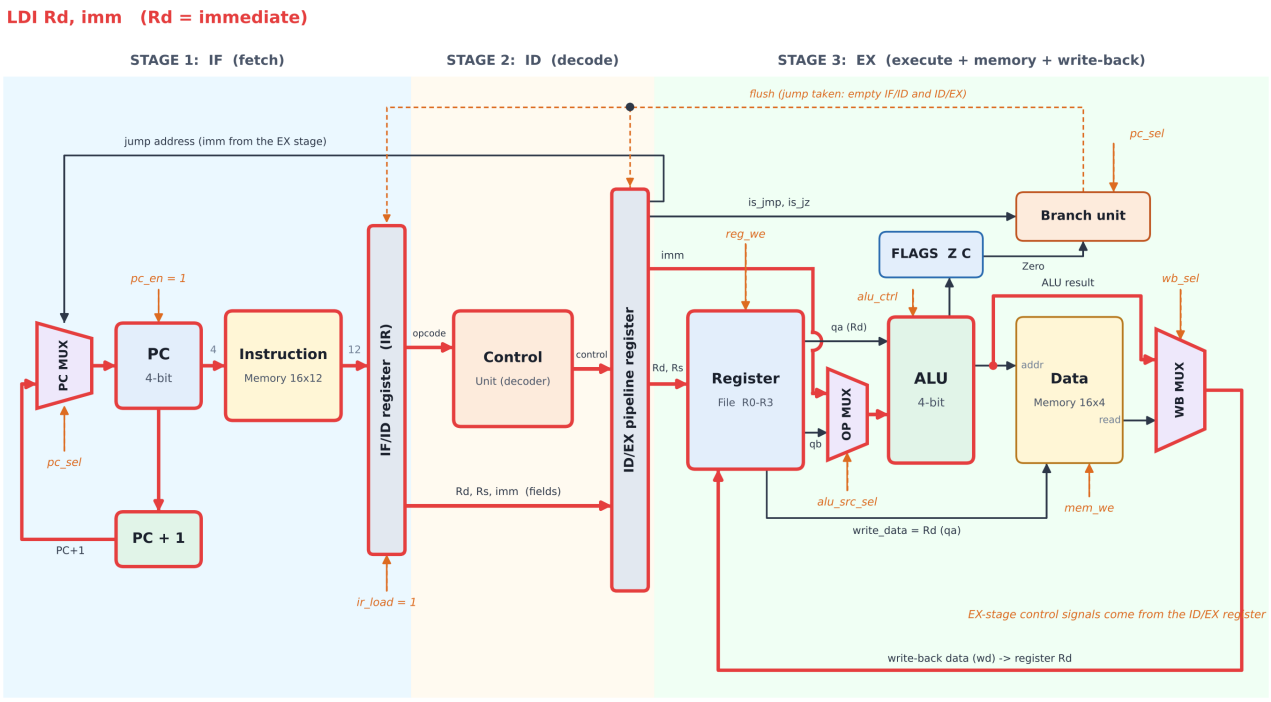


Figure 9. *Path of LDI Rd, imm.*

In EX the operand MUX passes the immediate (alu_src_sel = 1), the ALU copies it (PASS), and it is written into Rd. Flags do not change.

8.3 LOAD Rd, addr

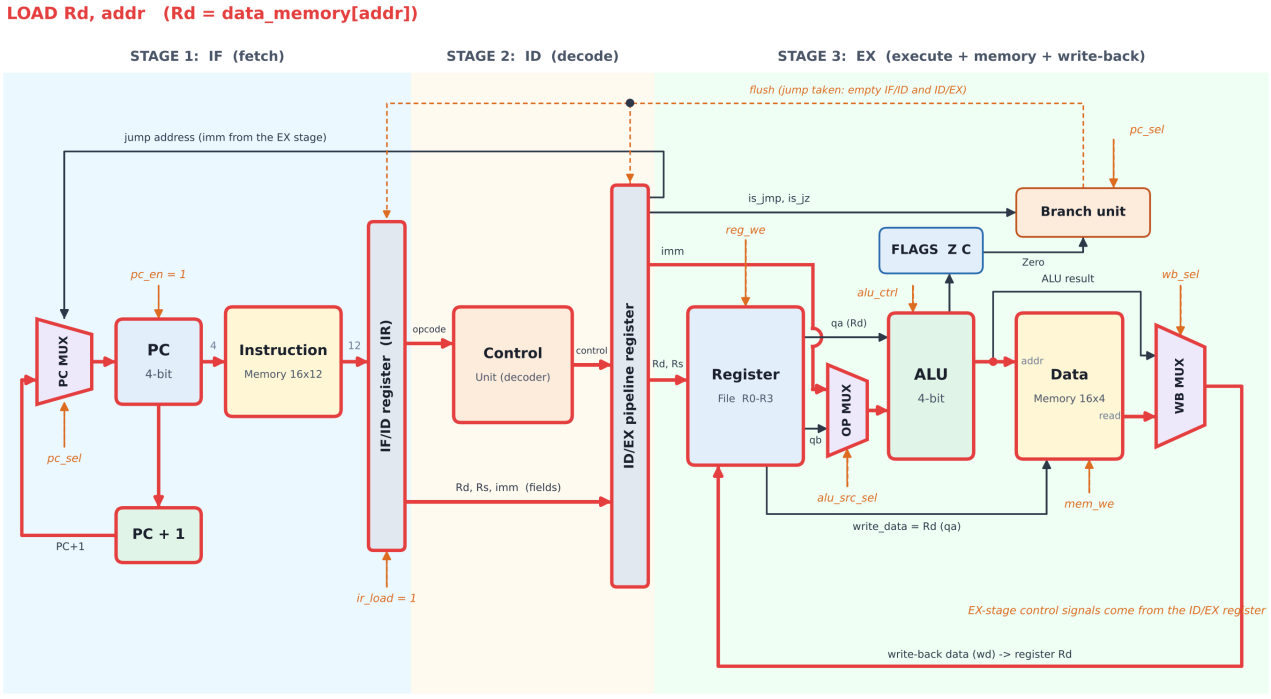


Figure 10. *Path of LOAD Rd, addr.*

In EX the immediate goes through the ALU (PASS) to the data-memory address; the memory gives the stored value; wb_sel = 1 sends it to the register file; at the clock edge Rd ← data_memory[addr].

8.4 STORE Rd, addr

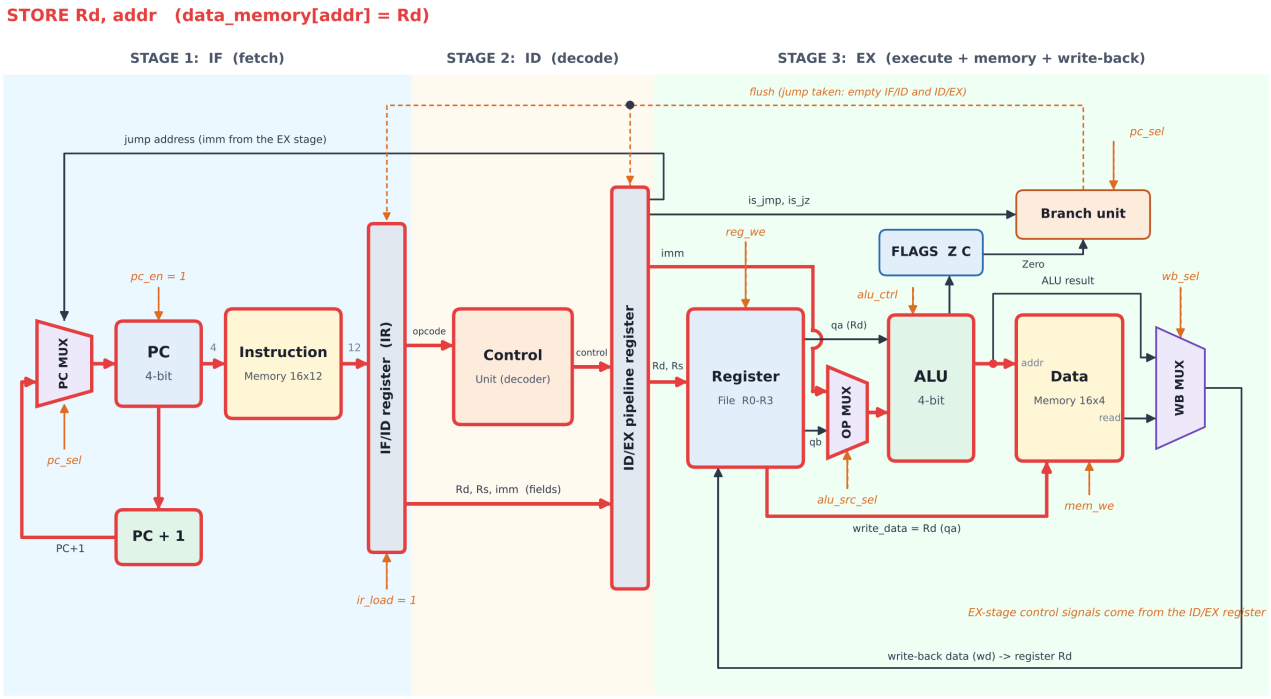


Figure 11. *Path of STORE Rd, addr.*

In EX the value of Rd goes to the memory's write data, the immediate goes through the ALU to the address, and mem_we = 1. At the clock edge data_memory[addr] ← Rd. No register changes.

8.5 JZ addr (taken)

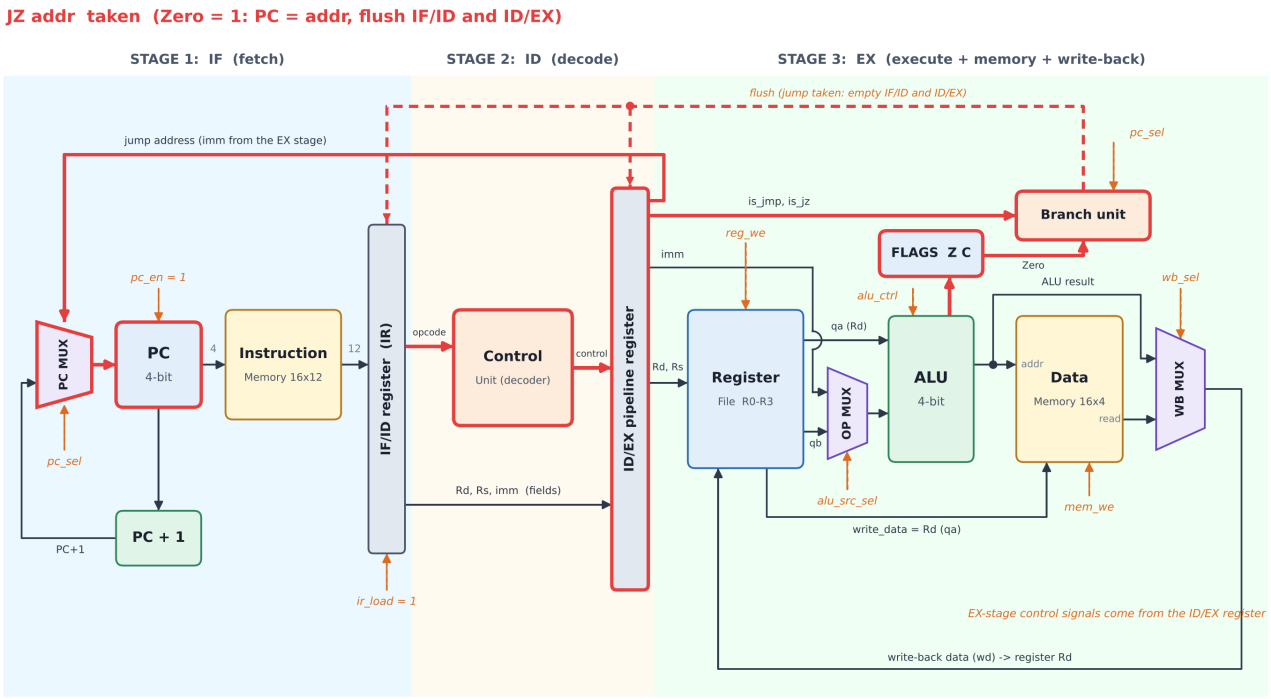


Figure 12. *JZ when Zero = 1: the PC loads the jump address and the two younger instructions are flushed.*

When JZ reaches EX, the branch unit reads the stored Zero flag. If it is 1, pc_sel = 1 and flush = 1: at the clock edge the PC loads the jump address and the IF/ID and ID/EX registers are emptied. If Zero is 0, nothing special happens and the pipeline simply continues.

Part B

Practical Implementation

Project plan, Verilog modules, programs, verification and results

9. Project Plan and Compliance

This chapter checks the design against the project plan given by the project seniors.

9.1 Requirements traceability

Plan requirement (Section 5)	How it is met	Verified by
Synchronous operation using a common clock	All sequential blocks use the same clk, rising edge	All testbenches
Reset places PC and control state in a known condition	PC = 0, registers = 0, flags = 0, both pipeline registers empty (NOP)	tb_cpu Test 5, tb_register_file
Register file: two combinational read ports, one clocked write port	register_file.v: qa, qb with always @(*), write on posedge clk	tb_register_file
ALU supports ADD, SUB, AND, OR, XOR, NOT	alu.v (adder_subtractor + logic_unit)	tb_alu, tb_cpu Tests 2-3
ALU generates Zero and a Carry / borrow flag	zero_detector; carry = carry-out / no-borrow	tb_alu, tb_cpu Test 2
Control unit decodes every opcode deterministically	instruction_decoder with safe defaults; reserved opcodes = NOP	tb_control_unit (all 16 opcodes)
PC increments and supports jump / conditional branch	PC MUX + branch unit (JMP, JZ) + flush	tb_control_unit, tb_cpu Test 4
CPU executes a test program and produces expected results	4 programs (.mem files)	tb_cpu Tests 1-4

Table 20. *Functional requirements and where they are met*

9.2 Module plan and interfaces

Module / file	Plan interface	Implemented ports
alu.v	a, b, alu_ctrl, result, carry, zero	same
register_file.v	clk, rst, ra, rb, wa, wd, we, qa, qb	same
program_counter.v	clk, rst, pc_en, pc_next, pc_out	same
instruction_memory.v	address, instruction	same
instruction_register.v	clk, rst, ir_load, instruction_in, instruction_out	same + flush, pc_in, pc_out, valid_out (pipeline)
control_unit.v	opcode, flags, alu_ctrl, reg_we, mux_sel, pc controls	opcode, zero_flag, alu_ctrl, reg_we, mem_we, alu_src_sel, wb_sel, zero_we, carry_we, is_jmp, is_jz, pc_sel, flush
data_memory.v	clk, address, write_data, mem_we, read_data	same
cpu_top.v	clk, rst, observable debug outputs	clk, rst, pc_out, instr_out, alu_out, wb_data_out, zero_out, carry_out
tb_cpu.v	stimulus, checks, pass/fail reporting	4 programs + reset test, 44 checks, PASS/FAIL summary

Table 21. *Module interfaces: project plan versus implementation*

Extra files not listed separately in the plan: `pipeline_register.v` (the ID/EX register), `mux.v` (the three multiplexers), `flag_register.v`, and three module-level testbenches.

9.3 Design decisions and differences from the plan

Topic	Plan	This implementation	Reason
Execution model	Single-cycle baseline	3-stage pipeline (IF, ID, EX)	Real fetch into a clocked IR and a separate decode stage, while still finishing one instruction per clock (Section 3.7)
Branch handling	PC loads branch target	Decided in EX; 2 instructions flushed	Simple and always correct; costs 2 cycles per taken jump
Data memory	Optional	Included	Lets LOAD and STORE be built and tested
Carry after SUB	Must be documented	Carry = 1 means no borrow	Same convention in ALU, flags and tests
Memory address	"ALU address"	ALU passes the immediate to the address	Same path as LDI
Reserved opcodes	Unused	Behave as NOP	Known decode result for every opcode

Table 22. *Design decisions*

9.4 Two-week schedule

Week	Work package	Activities	Milestone / evidence
1	Requirements and ISA	Scope, instruction format, opcode table, reset, flags, test strategy	Chapters 1, 6
1	Datapath design	Block diagram; buses, register file, ALU, MUX, PC, memory and pipeline registers	Datapath schematic (Chapter 8), interface table
1	ALU and registers	Code ALU and register file; module-level testbenches	tb_alu and tb_register_file pass
2	Control and fetch path	PC, instruction memory, IR, decoder, branch unit	tb_control_unit passes
2	Integration	Connect stages in cpu_top; pipeline registers; data memory; run programs	tb_cpu Tests 1-4 pass
2	Verification and documentation	Regression, waveforms, pipeline charts, report, optional FPGA demo	88 / 88 checks; this report

Table 23. *Project schedule*

9.5 Risks and mitigation

Risk	Possible effect	What was done
Instruction fields do not fit	Unclear decoding	12-bit format fixed before coding
Control / datapath mismatch	Wrong operation or write	Decoder truth table checked opcode by opcode
Carry / borrow ambiguity	Inconsistent flags	One convention (no-borrow) everywhere
Pipeline hazards	Wrong value read, or wrong instruction executed after a jump	Register read in EX (no data hazard); flush after a taken jump; both tested in tb_cpu
Uninitialised memory / registers	X values in simulation	Reset; memories initialised; X check in tb_cpu
Integration delays	Late interface errors	Plan interfaces fixed; module tests first
Scope expansion	Running out of time	FPGA demo and extra branches kept optional

Table 24. *Risks and mitigation*

10. Project Structure and Coding Style

10.1 Tools

- **Xilinx Vivado** (behavioural simulation with XSim; optional synthesis and FPGA).
- **Icarus Verilog** was used to run every testbench while preparing this report; the files run unchanged in Vivado and ModelSim / Questa (plain Verilog-2005, no SystemVerilog).
- Any text editor for the Verilog and `.mem` files.

10.2 Coding style

- **One job per module.** Larger blocks are split into small sub-modules in the same file: `alu.v` has adder_subtractor, logic_unit and zero_detector; `control_unit.v` has instruction_decoder and branch_unit.
- **Plain if/else and case statements**, no clever tricks; the four registers are written as `r0-r3`.
- **A comment on almost every line**, and a header in every file that lists its interface.
- **cpu_top.v is written stage by stage**: STAGE 1 IF, STAGE 2 ID, STAGE 3 EX, with signal names starting with `id_` or ex_ to show which stage they belong to.

10.3 Module hierarchy and files

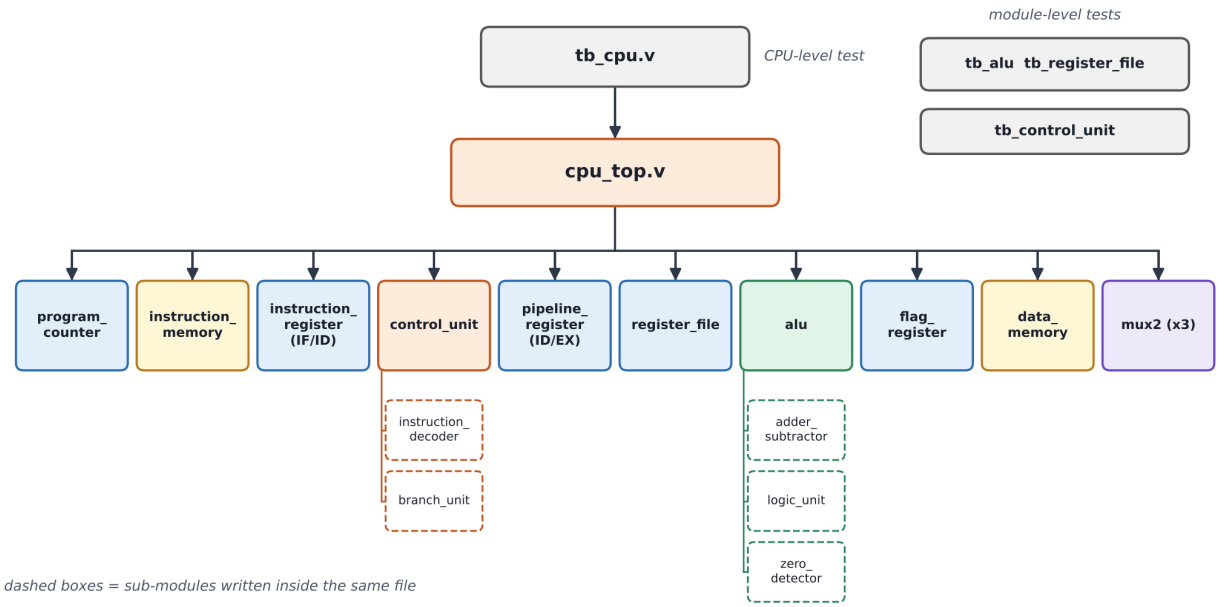


Figure 13. *Module hierarchy and testbenches. Dashed boxes are sub-modules inside alu.v and control_unit.v.*

File	Contents
alu.v	adder_subtractor, logic_unit, zero_detector, alu
register_file.v	register_file (R0-R3)
program_counter.v	program_counter
instruction_memory.v	instruction_memory (16 x 12, loads a .mem file)
instruction_register.v	instruction_register = IF/ID pipeline register
pipeline_register.v	pipeline_register = ID/EX pipeline register
control_unit.v	instruction_decoder, branch_unit, control_unit
data_memory.v	data_memory (16 x 4)
mux.v	mux2 (used 3 times)
flag_register.v	flag_register (Zero, Carry)
cpu_top.v	cpu_top: connects everything, stage by stage
tb_alu.v, tb_register_file.v, tb_control_unit.v	module-level testbenches
tb_cpu.v	CPU-level testbench
program.mem	demonstration program (plan Appendix A)
prog_arith.mem, prog_logic.mem, prog_jump.mem	test programs used by tb_cpu

Table 25. *Project files*

11. Module Implementation

This chapter gives the full Verilog code of every module, each followed by a short explanation. The files are also supplied separately in the project folder.

11.1 ALU (alu.v)

alu.v

```
// ============================================================================
// FILE: alu.v
// The ALU (Arithmetic Logic Unit) does all the calculations.
//
// Interface (as in the project plan):
//   a[3:0], b[3:0], alu_ctrl, result[3:0], carry, zero
//
// This file contains 4 small modules:
//   1. adder_subtractor : does ADD and SUB, gives the carry
//   2. logic_unit       : does AND, OR, XOR, NOT and PASS (copy b)
//   3. zero_detector    : checks if the result is 0000
//   4. alu              : the main ALU, it joins the 3 parts above
//
// ALU control codes (alu_ctrl):
//   0 = ADD    1 = SUB    2 = AND    3 = OR
//   4 = XOR    5 = NOT    6 = PASS (result = b; used by LDI, MOV,
//                                   LOAD and STORE)
//
// CARRY CONVENTION (same in ALU, flag register and testbenches):
//   ADD : carry = 1 when the sum is bigger than 15 (carry-out)
//   SUB : carry = 1 when a >= b, i.e. NO BORROW
//         carry = 0 when a <  b, i.e. a borrow happened
//
// CONNECTIONS
//   Calls       : only modules inside this same file:
//                 adder_subtractor, logic_unit, zero_detector
//   Called by   : cpu_top.v - instance ALU
//                 tb_alu.v - instance DUT
// ============================================================================
// ----------------------------------------------------------------------------
// MODULE 1: adder_subtractor
// ----------------------------------------------------------------------------
module adder_subtractor (
    input      [3:0] a,
    input      [3:0] b,
    input            do_sub,     // 0 = add, 1 = subtract
    output reg [3:0] result,
    output reg       carry
);
    reg [4:0] full_sum;          // 5 bits, so we can see the carry bit
    always @(*) begin
        full_sum = 5'b00000;
        if (do_sub == 1'b0) begin
            // ---- ADD ----
            full_sum = a + b;            // e.g. 15 + 1 = 16 = 1_0000
            result   = full_sum[3:0];    // lower 4 bits are the answer
            carry    = full_sum[4];      // 5th bit is the carry
        end
        else begin
            // ---- SUB ----
            result = a - b;              // e.g. 5 - 3 = 2
            if (a >= b)
                carry = 1'b1;            // no borrow
            else
                carry = 1'b0;            // borrow happened
        end
    end
endmodule
// ----------------------------------------------------------------------------
// MODULE 2: logic_unit
// ----------------------------------------------------------------------------
```

alu.v

```
module logic_unit (
    input      [3:0] a,
    input      [3:0] b,
    input      [2:0] alu_ctrl,
    output reg [3:0] result
);
    always @(*) begin
        case (alu_ctrl)
            3'd2:    result = a & b;     // AND
            3'd3:    result = a | b;     // OR
            3'd4:    result = a ^ b;     // XOR
            3'd5:    result = ~a;        // NOT (only uses a)
            3'd6:    result = b;         // PASS: copy b
            default: result = 4'b0000;
        endcase
    end
endmodule
// ------------------------------------------------------------------------
// MODULE 3: zero_detector
// ------------------------------------------------------------------------
module zero_detector (
    input  [3:0] value,
    output       zero
);
    assign zero = (value == 4'b0000) ? 1'b1 : 1'b0;
endmodule
// ------------------------------------------------------------------------
// MODULE 4: alu (main ALU)
// ------------------------------------------------------------------------
module alu (
    input      [3:0] a,          // operand A (register Rd)
    input      [3:0] b,          // operand B (register Rs or immediate)
    input      [2:0] alu_ctrl,   // which operation to do
    output reg [3:0] result,     // final answer
    output reg       carry,      // carry / no-borrow
    output           zero        // 1 when result = 0000
);
    wire [3:0] arith_result;     // answer from adder_subtractor
    wire       arith_carry;      // carry  from adder_subtractor
    wire [3:0] logic_result;     // answer from logic_unit
    wire do_sub = (alu_ctrl == 3'd1);    // alu_ctrl 1 means SUB
    // CALLS: module adder_subtractor (MODULE 1 in this same file, alu.v)
    adder_subtractor ADDSUB (
        .a(a), .b(b), .do_sub(do_sub),
        .result(arith_result), .carry(arith_carry)
    );
    // CALLS: module logic_unit (MODULE 2 in this same file, alu.v)
    logic_unit LOGIC (
        .a(a), .b(b), .alu_ctrl(alu_ctrl),
        .result(logic_result)
    );
    // choose the final answer
    always @(*) begin
        if (alu_ctrl == 3'd0 || alu_ctrl == 3'd1) begin
            result = arith_result;       // ADD or SUB
            carry  = arith_carry;
        end
        else begin
            result = logic_result;       // AND, OR, XOR, NOT, PASS
            carry  = 1'b0;               // logic ops have no carry
        end
    end
    // CALLS: module zero_detector (MODULE 3 in this same file, alu.v)
    zero_detector ZERO (
        .value(result),
        .zero(zero)
    );
endmodule
```

- **adder_subtractor:** ADD uses a 5-bit `full_sum` so bit 4 is the carry; SUB writes a - b and sets carry = 1 when a >= b (no borrow).

- **logic_unit:** one case statement for AND, OR, XOR, NOT and PASS.
- **zero_detector:** one comparison with 0000.
- **alu:** takes the adder's answer for ADD/SUB and the logic unit's answer otherwise. Every output gets a value in every branch, so no latches are created.

11.2 Register file (register_file.v)

register_file.v

```
// ======================================================================
// FILE: register_file.v
// 4 general-purpose registers (R0-R3), each 4 bits wide.
//
// Interface (as in the project plan):
//   clk, rst, ra, rb, wa, wd, we, qa, qb
//
//   ra, qa : read port A  (address in, value out)  - no clock needed
//   rb, qb : read port B  (address in, value out)  - no clock needed
//   wa, wd : write address and write data          - on clock edge
//   we     : write enable (1 = write)
//
// Register numbers: R0 = 00, R1 = 01, R2 = 10, R3 = 11
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance RF
//                 tb_register_file.v - instance DUT
//                 tb_cpu.v - reads CPU.RF.r0 ... CPU.RF.r3
// ======================================================================
module register_file (
    input            clk,
    input            rst,
    input      [1:0] ra,          // read address A
    input      [1:0] rb,          // read address B
    input      [1:0] wa,          // write address
    input      [3:0] wd,          // write data
    input            we,          // write enable
    output reg [3:0] qa,          // read data A
    output reg [3:0] qb           // read data B
);
    // the four registers
    reg [3:0] r0, r1, r2, r3;
    // ---------------- WRITE (clocked) ----------------
    always @(posedge clk) begin
        if (rst) begin
            r0 <= 4'b0000;        // reset clears all registers
            r1 <= 4'b0000;
            r2 <= 4'b0000;
            r3 <= 4'b0000;
        end
        else if (we) begin
            case (wa)
                2'd0: r0 <= wd;
                2'd1: r1 <= wd;
                2'd2: r2 <= wd;
                2'd3: r3 <= wd;
            endcase
        end
    end
    // ---------------- READ PORT A (combinational) ----------------
    always @(*) begin
        case (ra)
            2'd0: qa = r0;
            2'd1: qa = r1;
            2'd2: qa = r2;
            2'd3: qa = r3;
        endcase
    end
    // ---------------- READ PORT B (combinational) ----------------
    always @(*) begin
```

register_file.v

```
        case (rb)
            2'd0: qb = r0;
            2'd1: qb = r1;
            2'd2: qb = r2;
            2'd3: qb = r3;
        endcase
    end
endmodule
```

Writing is a clocked case statement on wa. The two read ports are separate `always @(*)` blocks, so qa and qb follow ra and rb straight away.

11.3 Program counter (program_counter.v)

program_counter.v

```
// ============================================================================
// FILE: program_counter.v
// The PC holds the address of the instruction being FETCHED (0 to 15).
//
// Interface (as in the project plan):
//   clk, rst, pc_en, pc_next, pc_out
//
// In the 3-stage pipeline a new instruction is fetched every clock,
// so pc_en is kept at 1 by cpu_top. pc_next is PC + 1, or the jump
// address when a jump is taken in the EX stage.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance PC
// ============================================================================
module program_counter (
    input            clk,
    input            rst,
    input            pc_en,       // 1 = load pc_next
    input      [3:0] pc_next,     // the next address
    output reg [3:0] pc_out       // current fetch address
);
    always @(posedge clk) begin
        if (rst)
            pc_out <= 4'd0;       // after reset, start at address 0
        else if (pc_en)
            pc_out <= pc_next;    // go to the next address
        // if pc_en = 0, pc_out keeps its old value
    end
endmodule
```

11.4 Instruction memory (instruction_memory.v)

instruction_memory.v

```
// ============================================================================
// FILE: instruction_memory.v
// Stores the program: 16 words x 12 bits (an initialised ROM).
//
// Interface (as in the project plan):
//   address[3:0], instruction[11:0]
//
// The program is read from a text file (default: program.mem).
// Each line of the file is one instruction in hex, e.g. 645
//
// CONNECTIONS
//   Calls       : nothing (built-in $readmemh only)
//   Called by   : cpu_top.v - instance IMEM
//                 tb_cpu.v - task load_program writes CPU.IMEM.mem
// ============================================================================
module instruction_memory #(
```

instruction_memory.v

```
    parameter PROGRAM_FILE = "program.mem"   // file with the program
)(
    input  [3:0]  address,        // from the PC
    output [11:0] instruction     // instruction at that address
);
    reg [11:0] mem [0:15];        // 16 words, 12 bits each
    integer i;
    initial begin
        // fill every word with C00 first (C = reserved opcode = NOP),
        // so an empty location never contains unknown (X) values
        for (i = 0; i < 16; i = i + 1)
            mem[i] = 12'hC00;
        // then load the program from the file
        $readmemh(PROGRAM_FILE, mem);
    end
    // read: output the instruction at the given address
    assign instruction = mem[address];
endmodule
```

PROGRAM_FILE is a parameter, so the same hardware can run different programs. All words are first set to C00 (a reserved opcode = NOP), so empty words never contain unknown values.

11.5 IF/ID pipeline register (instruction_register.v)

instruction_register.v

```
// ======================================================================
// FILE: instruction_register.v
// The IR is the IF/ID PIPELINE REGISTER: it sits between the
// FETCH stage and the DECODE stage.
//
// Interface (as in the project plan):
//   clk, rst, ir_load, instruction_in, instruction_out
// Extra pipeline signals:
//   flush           : 1 = throw the instruction away (taken jump)
//   pc_in / pc_out  : address of the instruction (to follow it in the
//                     pipeline and in the waveform)
//   valid_out       : 1 = holds a real instruction, 0 = empty (bubble)
//
// Every clock (ir_load = 1) it saves the instruction just fetched.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance IR (the IF/ID register)
// ======================================================================
module instruction_register (
    input             clk,
    input             rst,
    input             ir_load,          // 1 = save the new instruction
    input             flush,            // 1 = replace it with a NOP
    input      [11:0] instruction_in,   // from instruction memory
    input      [3:0]  pc_in,            // its address
    output reg [11:0] instruction_out,  // to the DECODE stage
    output reg [3:0]  pc_out,
    output reg        valid_out
);
    always @(posedge clk) begin
        if (rst || flush) begin
            instruction_out <= 12'hC00;     // C00 = reserved opcode = NOP
            pc_out          <= 4'd0;
            valid_out       <= 1'b0;        // empty slot (bubble)
        end
        else if (ir_load) begin
            instruction_out <= instruction_in;
            pc_out          <= pc_in;
            valid_out       <= 1'b1;
        end
    end
endmodule
```

Every clock it saves the instruction just fetched, its address and valid = 1. On reset or flush it is loaded with NOP and valid = 0 (an empty slot, a "bubble").

11.6 ID/EX pipeline register (pipeline_register.v)

pipeline_register.v

```
// ======================================================================
// FILE: pipeline_register.v
// The ID/EX PIPELINE REGISTER: it sits between the DECODE stage and
// the EXECUTE stage.
//
// At every clock it copies everything the EXECUTE stage will need:
//   - the control signals made by the decoder
//   - the instruction fields: Rd, Rs, imm
//   - the instruction address and a valid bit (for tracing)
//
// flush = 1 (taken jump) or rst = 1 loads a "do nothing" instruction:
// all control signals 0, so the EXECUTE stage changes nothing.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance ID_EX
// ======================================================================
module pipeline_register (
    input             clk,
    input             rst,
    input             flush,
    // ---- inputs from the DECODE stage ----
    input      [2:0]  alu_ctrl_in,
    input             alu_src_sel_in,
    input             wb_sel_in,
    input             reg_we_in,
    input             mem_we_in,
    input             zero_we_in,
    input             carry_we_in,
    input             is_jmp_in,
    input             is_jz_in,
    input      [1:0]  rd_in,
    input      [1:0]  rs_in,
    input      [3:0]  imm_in,
    input      [3:0]  pc_in,
    input             valid_in,
    input      [11:0] instr_in,
    // ---- outputs to the EXECUTE stage ----
    output reg [2:0]  alu_ctrl,
    output reg        alu_src_sel,
    output reg        wb_sel,
    output reg        reg_we,
    output reg        mem_we,
    output reg        zero_we,
    output reg        carry_we,
    output reg        is_jmp,
    output reg        is_jz,
    output reg [1:0]  rd,
    output reg [1:0]  rs,
    output reg [3:0]  imm,
    output reg [3:0]  pc,
    output reg        valid,
    output reg [11:0] instr
);
    always @(posedge clk) begin
        if (rst || flush) begin
            // ---- bubble: nothing will be written in EXECUTE ----
            alu_ctrl    <= 3'd0;
            alu_src_sel <= 1'b0;
            wb_sel      <= 1'b0;
            reg_we      <= 1'b0;
            mem_we      <= 1'b0;
            zero_we     <= 1'b0;
            carry_we    <= 1'b0;
```

pipeline_register.v

```
            is_jmp      <= 1'b0;
            is_jz       <= 1'b0;
            rd          <= 2'd0;
            rs          <= 2'd0;
            imm         <= 4'd0;
            pc          <= 4'd0;
            valid       <= 1'b0;
            instr       <= 12'hC00;
        end
        else begin
            // ---- pass the decoded instruction on to EXECUTE ----
            alu_ctrl    <= alu_ctrl_in;
            alu_src_sel <= alu_src_sel_in;
            wb_sel      <= wb_sel_in;
            reg_we      <= reg_we_in;
            mem_we      <= mem_we_in;
            zero_we     <= zero_we_in;
            carry_we    <= carry_we_in;
            is_jmp      <= is_jmp_in;
            is_jz       <= is_jz_in;
            rd          <= rd_in;
            rs          <= rs_in;
            imm         <= imm_in;
            pc          <= pc_in;
            valid       <= valid_in;
            instr       <= instr_in;
        end
    end
endmodule
```

A plain set of registers: *_in* signals from the ID stage are copied to the outputs for the EX stage at every clock. On reset or flush every control signal becomes 0, so the EX stage writes nothing.

11.7 Data memory (data_memory.v)

data_memory.v

```
// ======================================================================
// FILE: data_memory.v
// Data RAM: 16 words x 4 bits. Used by LOAD and STORE.
//
// Interface (as in the project plan):
//   clk, address, write_data, mem_we, read_data
//
// - Read  : any time (no clock needed)          -> LOAD
// - Write : on the clock edge when mem_we = 1    -> STORE
// It is separate from the instruction memory (Harvard organisation).
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance DMEM
//                 tb_cpu.v - reads and clears CPU.DMEM.mem
// ======================================================================
module data_memory (
    input        clk,
    input  [3:0] address,         // which word
    input  [3:0] write_data,      // data to write (STORE)
    input        mem_we,          // write enable: 1 = write
    output [3:0] read_data        // data read (LOAD)
);
    reg [3:0] mem [0:15];         // 16 words, 4 bits each
    integer i;
    // start with all words = 0 (no unknown X values)
    initial begin
        for (i = 0; i < 16; i = i + 1)
            mem[i] = 4'b0000;
    end
    // write on the clock edge
    always @(posedge clk) begin
```

data_memory.v

```
        if (mem_we)
            mem[address] <= write_data;
    end
    // read any time
    assign read_data = mem[address];
endmodule
```

11.8 Multiplexer (mux.v)

mux.v

```
// ============================================================================
// FILE: mux.v
// 2-input, 4-bit multiplexer. A mux picks ONE of its inputs.
//   sel = 0 -> y = d0
//   sel = 1 -> y = d1
//
// The CPU uses this module three times:
//   1. Operand MUX    : ALU input B = register Rs  or immediate
//   2. Write-back MUX : register data = ALU result or memory data
//   3. PC MUX         : next PC = PC + 1  or jump address
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - 3 times: instances PC_MUX, OPERAND_MUX, WB_MUX
// ============================================================================
module mux2 (
    input  [3:0] d0,
    input  [3:0] d1,
    input        sel,
    output [3:0] y
);
    assign y = (sel == 1'b1) ? d1 : d0;
endmodule
```

11.9 Flag register (flag_register.v)

flag_register.v

```
// ============================================================================
// FILE: flag_register.v
// Saves the Zero and Carry flags so a later instruction (JZ) can use
// them, and so they can be observed in the waveform.
//   zero  : last arithmetic/logic result was 0000
//   carry : carry-out from ADD, or NO-BORROW from SUB
// They have separate enables because AND/OR/XOR/NOT change only Zero,
// while ADD/SUB change both Zero and Carry.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance FLAGS
// ============================================================================
module flag_register (
    input      clk,
    input      rst,
    input      zero_we,     // 1 = save new zero
    input      carry_we,    // 1 = save new carry
    input      zero_in,     // from the ALU
    input      carry_in,    // from the ALU
    output reg zero_flag,   // saved zero
    output reg carry_flag   // saved carry
);
    always @(posedge clk) begin
        if (rst) begin
            zero_flag  <= 1'b0;
            carry_flag <= 1'b0;
        end
```

flag_register.v

```
        else begin
            if (zero_we)  zero_flag  <= zero_in;
            if (carry_we) carry_flag <= carry_in;
        end
    end
endmodule
```

11.10 Control unit (control_unit.v)

control_unit.v

```
// ============================================================================
// FILE: control_unit.v
// Hardwired control unit for the 3-stage pipeline.
//
// Interface (as in the project plan):
//   opcode, flags, alu_ctrl, reg_we, mux selects, PC controls
//
// This file contains 3 modules:
//   1. instruction_decoder : DECODE stage - turns the opcode into
//                            control signals (pure logic, no clock)
//   2. branch_unit         : EXECUTE stage - decides if a jump is taken,
//                            and if so asks the pipeline to flush
//   3. control_unit        : joins the two modules above
//
// CONNECTIONS
//   Calls       : only modules inside this same file:
//                 instruction_decoder, branch_unit
//   Called by   : cpu_top.v - instance CU
//                 tb_control_unit.v - instance DUT
// ============================================================================
// ----------------------------------------------------------------------------
// MODULE 1: instruction_decoder  (used in the DECODE stage)
// Turns ON only the signals an instruction needs.
// ----------------------------------------------------------------------------
module instruction_decoder (
    input      [3:0] opcode,      // instruction bits [11:8]
    output reg [2:0] alu_ctrl,    // ALU operation
    output reg       alu_src_sel, // operand MUX: 0 = Rs, 1 = immediate
    output reg       wb_sel,      // write-back MUX: 0 = ALU, 1 = memory
    output reg       reg_we,      // 1 = write register Rd
    output reg       mem_we,      // 1 = write data memory
    output reg       zero_we,     // 1 = update Zero flag
    output reg       carry_we,    // 1 = update Carry flag
    output reg       is_jmp,      // 1 = this is JMP
    output reg       is_jz        // 1 = this is JZ
);
    // ---- opcodes (project plan, Section 4) ----
    localparam ADD   = 4'b0000;
    localparam SUB   = 4'b0001;
    localparam AND   = 4'b0010;
    localparam OR    = 4'b0011;
    localparam XOR   = 4'b0100;
    localparam NOT   = 4'b0101;
    localparam LDI   = 4'b0110;
    localparam MOV   = 4'b0111;
    localparam LOAD  = 4'b1000;
    localparam STORE = 4'b1001;
    localparam JMP   = 4'b1010;
    localparam JZ    = 4'b1011;
    // 1100 - 1111 are reserved: they do nothing (NOP)
    // ---- ALU control codes (must match alu.v) ----
    localparam ALU_ADD  = 3'd0;
    localparam ALU_SUB  = 3'd1;
    localparam ALU_AND  = 3'd2;
    localparam ALU_OR   = 3'd3;
    localparam ALU_XOR  = 3'd4;
    localparam ALU_NOT  = 3'd5;
    localparam ALU_PASS = 3'd6;
    always @(*) begin
```

control_unit.v

```
        // ---- Step 1: default = do nothing ----
        alu_ctrl    = ALU_ADD;
        alu_src_sel = 1'b0;
        wb_sel      = 1'b0;
        reg_we      = 1'b0;
        mem_we      = 1'b0;
        zero_we     = 1'b0;
        carry_we    = 1'b0;
        is_jmp      = 1'b0;
        is_jz       = 1'b0;
        // ---- Step 2: turn on what the opcode needs ----
        case (opcode)
            ADD: begin                    // Rd = Rd + Rs
                alu_ctrl = ALU_ADD;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
                carry_we = 1'b1;
            end
            SUB: begin                    // Rd = Rd - Rs
                alu_ctrl = ALU_SUB;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
                carry_we = 1'b1;
            end
            AND: begin                    // Rd = Rd AND Rs
                alu_ctrl = ALU_AND;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            OR: begin                     // Rd = Rd OR Rs
                alu_ctrl = ALU_OR;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            XOR: begin                    // Rd = Rd XOR Rs
                alu_ctrl = ALU_XOR;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            NOT: begin                    // Rd = NOT Rd
                alu_ctrl = ALU_NOT;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            LDI: begin                    // Rd = immediate
                alu_ctrl    = ALU_PASS;   // ALU passes operand B
                alu_src_sel = 1'b1;       // operand B = immediate
                reg_we      = 1'b1;
            end
            MOV: begin                    // Rd = Rs
                alu_ctrl = ALU_PASS;      // ALU passes operand B = Rs
                reg_we   = 1'b1;
            end
            LOAD: begin                   // Rd = data_memory[addr]
                alu_ctrl    = ALU_PASS;   // ALU passes addr to memory
                alu_src_sel = 1'b1;
                wb_sel      = 1'b1;       // write memory data into Rd
                reg_we      = 1'b1;
            end
            STORE: begin                  // data_memory[addr] = Rd
                alu_ctrl    = ALU_PASS;   // ALU passes addr to memory
                alu_src_sel = 1'b1;
                mem_we      = 1'b1;
            end
            JMP: begin                    // PC = addr
                is_jmp = 1'b1;
            end
            JZ: begin                     // if Zero = 1, PC = addr
                is_jz = 1'b1;
            end
            default: begin
                // reserved opcodes 1100-1111: do nothing
            end
```

control_unit.v

```
        endcase
    end
endmodule
// ------------------------------------------------------------------------
// MODULE 2: branch_unit  (used in the EXECUTE stage)
// pc_sel = 1 means "jump": the PC loads the jump address, and the two
// younger instructions already in IF and ID are flushed (made NOP).
// ------------------------------------------------------------------------
module branch_unit (
    input      is_jmp,        // instruction in EX is JMP
    input      is_jz,         // instruction in EX is JZ
    input      zero_flag,     // saved Zero flag
    output reg pc_sel,        // PC MUX: 0 = PC + 1, 1 = jump address
    output     flush          // 1 = empty the IF/ID and ID/EX registers
);
    always @(*) begin
        if (is_jmp)
            pc_sel = 1'b1;                // JMP always jumps
        else if (is_jz && zero_flag)
            pc_sel = 1'b1;                // JZ jumps only when Zero = 1
        else
            pc_sel = 1'b0;                // otherwise: next address
    end
    assign flush = pc_sel;                // jump taken -> flush
endmodule
// ------------------------------------------------------------------------
// MODULE 3: control_unit (instruction_decoder + branch_unit)
// The decoder works on the instruction in DECODE; the branch unit
// works on the instruction in EXECUTE.
// ------------------------------------------------------------------------
module control_unit (
    // ---- DECODE stage ----
    input      [3:0] opcode,
    output     [2:0] alu_ctrl,
    output           alu_src_sel,
    output           wb_sel,
    output           reg_we,
    output           mem_we,
    output           zero_we,
    output           carry_we,
    output           is_jmp,
    output           is_jz,
    // ---- EXECUTE stage ----
    input            ex_is_jmp,
    input            ex_is_jz,
    input            zero_flag,
    output           pc_sel,
    output           flush
);
    // CALLS: module instruction_decoder (MODULE 1 in this same file, control_unit.v)
    instruction_decoder DECODER (
        .opcode      (opcode),
        .alu_ctrl    (alu_ctrl),
        .alu_src_sel (alu_src_sel),
        .wb_sel      (wb_sel),
        .reg_we      (reg_we),
        .mem_we      (mem_we),
        .zero_we     (zero_we),
        .carry_we    (carry_we),
        .is_jmp      (is_jmp),
        .is_jz       (is_jz)
    );
    // CALLS: module branch_unit (MODULE 2 in this same file, control_unit.v)
    branch_unit BRANCH (
        .is_jmp    (ex_is_jmp),
        .is_jz     (ex_is_jz),
        .zero_flag (zero_flag),
        .pc_sel    (pc_sel),
        .flush     (flush)
    );
endmodule
```

- **instruction_decoder Step 1:** every signal is set to 0. **Step 2:** the case statement turns on what the opcode needs. Opcodes have readable names made with `localparam`.
- **LDI, LOAD, STORE** set `alu_src_sel = 1` and `alu_ctrl = PASS`, so the immediate travels through the ALU.
- **branch_unit:** an if / else if / else that sets pc_sel for JMP, or for JZ when Zero = 1; `flush = pc_sel`.
- **control_unit:** joins the two. Its decoder ports connect to the ID stage and its branch ports to the EX stage.

11.11 Top level (cpu_top.v)

Part of cpu_top.v	Blocks used	What happens
STAGE 1: IF	mux2 (PC MUX), program_counter, instruction_memory, instruction_register	fetch the instruction at PC into IF/ID; PC ← PC + 1 or jump address
STAGE 2: ID	control_unit (decoder), pipeline_register	decode the opcode; save control + fields in ID/EX
STAGE 3: EX	register_file, mux2 (operand), alu, flag_register, data_memory, mux2 (write-back), control_unit (branch unit)	read registers, compute, access memory, write back, update flags, decide jumps

Table 26. *The three stages in cpu_top.v*

cpu_top.v

```
// ======================================================================
// FILE: cpu_top.v
// The complete 4-bit RISC CPU with a 3-STAGE PIPELINE.
// This file only CONNECTS the blocks.
//
//   STAGE 1  IF (fetch)   : PC -> instruction memory -> IF/ID register (IR)
//   STAGE 2  ID (decode)  : control unit decodes -> ID/EX register
//   STAGE 3  EX (execute) : read registers -> operand MUX -> ALU
//                           -> data memory -> write-back MUX -> Rd,
//                           update flags, decide jumps
//
// All three stages work at the same time on three different
// instructions, so one instruction can finish every clock.
// A taken jump (decided in EX) flushes the 2 younger instructions.
//
// Instruction format (12 bits):
//   [11:8] opcode | [7:6] Rd | [5:4] Rs | [3:0] immediate / address
//
// CONNECTIONS
//   Calls       : mux.v, program_counter.v, instruction_memory.v,
//                 instruction_register.v, control_unit.v, pipeline_register.v,
//                 register_file.v, alu.v, flag_register.v, data_memory.v
//   Called by   : tb_cpu.v - instance CPU
// ======================================================================
module cpu_top #(
    parameter PROGRAM_FILE = "program.mem"   // program to run
)(
    input         clk,
    input         rst,
    // observable debug outputs (waveform viewer or FPGA LEDs)
    output [3:0]  pc_out,          // fetch address (PC)
    output [11:0] instr_out,       // instruction in the EXECUTE stage
    output [3:0]  alu_out,         // ALU result
    output [3:0]  wb_data_out,     // value written to the destination register
    output        zero_out,        // Zero flag
    output        carry_out        // Carry flag
);
    // wires shared between stages
```

cpu_top.v

```
    wire        pc_sel;           // from the branch unit (EX)
    wire        flush;            // from the branch unit (EX)
    wire [3:0]  ex_imm;           // jump address (EX)
    // ================================================================
    // STAGE 1: IF - FETCH
    // ================================================================
    wire [3:0]  pc;               // fetch address
    wire [3:0]  pc_plus_1 = pc + 4'd1;
    wire [3:0]  pc_next;
    wire [11:0] fetched;          // word out of instruction memory
    // PC MUX: next address = PC + 1, or the jump address from EX
    // CALLS: module mux2 from file mux.v
    mux2 PC_MUX (
        .d0  (pc_plus_1),
        .d1  (ex_imm),
        .sel (pc_sel),
        .y   (pc_next)
    );
    // CALLS: module program_counter from file program_counter.v
    program_counter PC (
        .clk     (clk),
        .rst     (rst),
        .pc_en   (1'b1),          // fetch a new instruction every clock
        .pc_next (pc_next),
        .pc_out  (pc)
    );
    // CALLS: module instruction_memory from file instruction_memory.v
    instruction_memory #(.PROGRAM_FILE(PROGRAM_FILE)) IMEM (
        .address     (pc),
        .instruction (fetched)
    );
    // ---------------- IF/ID pipeline register (the IR) ----------------
    wire [11:0] id_instr;         // instruction now in DECODE
    wire [3:0]  id_pc;
    wire        id_valid;
    // CALLS: module instruction_register from file instruction_register.v
    instruction_register IR (
        .clk             (clk),
        .rst             (rst),
        .ir_load         (1'b1),   // load every clock
        .flush           (flush),
        .instruction_in  (fetched),
        .pc_in           (pc),
        .instruction_out (id_instr),
        .pc_out          (id_pc),
        .valid_out       (id_valid)
    );
    // ================================================================
    // STAGE 2: ID - DECODE
    // ================================================================
    wire [3:0] id_opcode = id_instr[11:8];
    wire [1:0] id_rd     = id_instr[7:6];
    wire [1:0] id_rs     = id_instr[5:4];
    wire [3:0] id_imm    = id_instr[3:0];
    wire [2:0] id_alu_ctrl;
    wire       id_alu_src_sel, id_wb_sel, id_reg_we, id_mem_we;
    wire       id_zero_we, id_carry_we, id_is_jmp, id_is_jz;
    wire       ex_is_jmp, ex_is_jz;   // from ID/EX (below)
    wire       zero_flag, carry_flag; // from the flag register (EX)
    // CALLS: module control_unit from file control_unit.v
    control_unit CU (
        // decode part
        .opcode      (id_opcode),
        .alu_ctrl    (id_alu_ctrl),
        .alu_src_sel (id_alu_src_sel),
        .wb_sel      (id_wb_sel),
        .reg_we      (id_reg_we),
        .mem_we      (id_mem_we),
        .zero_we     (id_zero_we),
        .carry_we    (id_carry_we),
        .is_jmp      (id_is_jmp),
        .is_jz       (id_is_jz),
        // branch part (EX stage)
```

cpu_top.v

```
        .ex_is_jmp   (ex_is_jmp),
        .ex_is_jz    (ex_is_jz),
        .zero_flag   (zero_flag),
        .pc_sel      (pc_sel),
        .flush       (flush)
    );
    // ---------------- ID/EX pipeline register ----------------
    wire [2:0]  ex_alu_ctrl;
    wire        ex_alu_src_sel, ex_wb_sel, ex_reg_we, ex_mem_we;
    wire        ex_zero_we, ex_carry_we;
    wire [1:0]  ex_rd, ex_rs;
    wire [3:0]  ex_pc;
    wire        ex_valid;
    wire [11:0] ex_instr;
    // CALLS: module pipeline_register from file pipeline_register.v
    pipeline_register ID_EX (
        .clk            (clk),
        .rst            (rst),
        .flush          (flush),
        .alu_ctrl_in    (id_alu_ctrl),
        .alu_src_sel_in (id_alu_src_sel),
        .wb_sel_in      (id_wb_sel),
        .reg_we_in      (id_reg_we),
        .mem_we_in      (id_mem_we),
        .zero_we_in     (id_zero_we),
        .carry_we_in    (id_carry_we),
        .is_jmp_in      (id_is_jmp),
        .is_jz_in       (id_is_jz),
        .rd_in          (id_rd),
        .rs_in          (id_rs),
        .imm_in         (id_imm),
        .pc_in          (id_pc),
        .valid_in       (id_valid),
        .instr_in       (id_instr),
        .alu_ctrl       (ex_alu_ctrl),
        .alu_src_sel    (ex_alu_src_sel),
        .wb_sel         (ex_wb_sel),
        .reg_we         (ex_reg_we),
        .mem_we         (ex_mem_we),
        .zero_we        (ex_zero_we),
        .carry_we       (ex_carry_we),
        .is_jmp         (ex_is_jmp),
        .is_jz          (ex_is_jz),
        .rd             (ex_rd),
        .rs             (ex_rs),
        .imm            (ex_imm),
        .pc             (ex_pc),
        .valid          (ex_valid),
        .instr          (ex_instr)
    );
    // ================================================================
    // STAGE 3: EX - EXECUTE (operand read, ALU, memory, write-back)
    // ================================================================
    wire [3:0] rd_value;           // qa: value of Rd
    wire [3:0] rs_value;           // qb: value of Rs
    wire [3:0] alu_b;
    wire [3:0] alu_result;
    wire       alu_zero, alu_carry;
    wire [3:0] mem_data;
    wire [3:0] wb_data;
    // ---- operand read ----
    // CALLS: module register_file from file register_file.v
    register_file RF (
        .clk (clk),
        .rst (rst),
        .ra  (ex_rd),              // port A reads Rd
        .rb  (ex_rs),              // port B reads Rs
        .wa  (ex_rd),              // result is written into Rd
        .wd  (wb_data),
        .we  (ex_reg_we),
        .qa  (rd_value),
        .qb  (rs_value)
    );
```

cpu_top.v

```
    // ---- operand MUX: B = Rs value (0) or immediate (1) ----
    // CALLS: module mux2 from file mux.v
    mux2 OPERAND_MUX (
        .d0  (rs_value),
        .d1  (ex_imm),
        .sel (ex_alu_src_sel),
        .y   (alu_b)
    );
    // ---- ALU ----
    // CALLS: module alu from file alu.v
    alu ALU (
        .a        (rd_value),
        .b        (alu_b),
        .alu_ctrl (ex_alu_ctrl),
        .result   (alu_result),
        .carry    (alu_carry),
        .zero     (alu_zero)
    );
    // ---- flags ----
    // CALLS: module flag_register from file flag_register.v
    flag_register FLAGS (
        .clk        (clk),
        .rst        (rst),
        .zero_we    (ex_zero_we),
        .carry_we   (ex_carry_we),
        .zero_in    (alu_zero),
        .carry_in   (alu_carry),
        .zero_flag  (zero_flag),
        .carry_flag (carry_flag)
    );
    // ---- memory access: address = ALU result, data = Rd ----
    // CALLS: module data_memory from file data_memory.v
    data_memory DMEM (
        .clk        (clk),
        .address    (alu_result),
        .write_data (rd_value),
        .mem_we     (ex_mem_we),
        .read_data  (mem_data)
    );
    // ---- write-back MUX: ALU result (0) or memory data (1) ----
    // CALLS: module mux2 from file mux.v
    mux2 WB_MUX (
        .d0  (alu_result),
        .d1  (mem_data),
        .sel (ex_wb_sel),
        .y   (wb_data)
    );
    // ================================================================
    // Debug outputs
    // ================================================================
    assign pc_out      = pc;
    assign instr_out   = ex_instr;
    assign alu_out     = alu_result;
    assign wb_data_out = wb_data;
    assign zero_out    = zero_flag;
    assign carry_out   = carry_flag;
endmodule
```

12. Programs

12.1 Writing a program by hand

Programs are written as hexadecimal machine code in a `.mem` file, one 3-digit word per line from address 0. Look up the opcode (Section 6.2), write Rd and Rs as 2-bit numbers (unused fields = 00), add the 4-bit immediate or address, then turn each group of 4 bits into one hex digit. Unused words are C00 (NOP).

12.2 Demonstration program (program.mem, plan Appendix A)

Addr	Hex	Instruction	Purpose
0	645	LDI R1, 0101	Load decimal 5 into R1
1	683	LDI R2, 0011	Load decimal 3 into R2
2	060	ADD R1, R2	R1 becomes 1000 (decimal 8)
3	6CA	LDI R3, 1010	Load a bit pattern into R3
4	470	XOR R1, R3	R1 becomes 0010 (decimal 2)
5	A05	JMP 0101	Loop at the last instruction for a stable demonstration

Table 27. *Demonstration program*

12.3 Test program: arithmetic and memory (prog_arith.mem)

Addr	Hex	Instruction	Effect
0	645	LDI R1, 5	R1 = 0101
1	683	LDI R2, 3	R2 = 0011
2	060	ADD R1, R2	R1 = 1000, Carry = 0, Zero = 0
3	940	STORE R1, 0	data_memory[0] = 1000
4	645	LDI R1, 5	R1 = 0101
5	160	SUB R1, R2	R1 = 0010, Zero = 0, Carry = 1 (no borrow)
6	941	STORE R1, 1	data_memory[1] = 0010
7	800	LOAD R0, 0	R0 = 1000 (reads what STORE wrote 4 instructions earlier)
8	1A0	SUB R2, R2	R2 = 0000, Zero = 1
9	A09	JMP 9	Stop

Table 28. *Arithmetic and memory test program*

12.4 Test program: logic operations (prog_logic.mem)

Addr	Hex	Instruction	Effect
0	64C	LDI R1, 1100	R1 = 1100
1	68A	LDI R2, 1010	R2 = 1010
2	7D0	MOV R3, R1	R3 = 1100 (uses R1 written just one instruction before)
3	2E0	AND R3, R2	R3 = 1000
4	7D0	MOV R3, R1	R3 = 1100
5	3E0	OR R3, R2	R3 = 1110
6	460	XOR R1, R2	R1 = 0110
7	790	MOV R2, R1	R2 = 0110
8	605	LDI R0, 0101	R0 = 0101
9	500	NOT R0	R0 = 1010
10	6CA	LDI R3, 1010	R3 = 1010
11	4F0	XOR R3, R3	R3 = 0000, Zero = 1
12	A0C	JMP 12	Stop

Table 29. *Logic test program*

12.5 Test program: PC and jumps (prog_jump.mem)

Addr	Hex	Instruction	Effect
0	601	LDI R0, 1	R0 = 1; PC 0 → 1 (increment)
1	A04	JMP 4	PC becomes 0100; addresses 2 and 3 are flushed
2-3	60F	LDI R0, 15	must never run
4	647	LDI R1, 7	R1 = 0111
5	150	SUB R1, R1	R1 = 0000, Zero = 1
6	B09	JZ 9	taken: PC = 9; addresses 7 and 8 are flushed
7-8	68F	LDI R2, 15	must never run
9	6C2	LDI R3, 2	R3 = 0010
10	0F0	ADD R3, R3	R3 = 0100, Zero = 0
11	B0E	JZ 14	not taken: continue with 12
12	686	LDI R2, 6	R2 = 0110
13	A0D	JMP 13	Stop
14	68F	LDI R2, 15	must never run

Table 30. *PC and jump test program*

13. Verification

As the project plan asks, testing is done at **two levels**: module-level tests that check each block on its own, and CPU-level tests that run programs from instruction memory. Every testbench is self-checking: it compares results with expected values and prints PASS or FAIL, then a final summary.

Level	Testbench	What it checks	Checks
Module	`tb_alu.v`	All ALU operations, Zero, Carry / no-borrow	11
Module	`tb_register_file.v`	Reset, write and read of every register, both ports, we = 0	11
Module	`tb_control_unit.v`	Decoder output for all 16 opcodes; branch unit in all 6 situations	22
CPU	`tb_cpu.v`	Demo program, arithmetic, memory, logic, MOV, LDI, Zero, PC increment, jumps and flushing, reset, no X values	44
		Total	88

Table 31. *Testbenches*

13.1 Test plan coverage (project plan Section 8)

Plan test	Setup	Expected	Where tested	Result
ADD	R1 = 0101, R2 = 0011; ADD R1,R2	R1 = 1000, Carry 0, Zero 0	tb_alu, tb_cpu T2	PASS
SUB	R1 = 0101, R2 = 0011; SUB R1,R2	R1 = 0010, Zero 0 (Carry 1)	tb_alu, tb_cpu T2	PASS
AND	1100 AND 1010	1000	tb_alu, tb_cpu T3	PASS
OR	1100 OR 1010	1110	tb_alu, tb_cpu T3	PASS
XOR	1100 XOR 1010	0110	tb_alu, tb_cpu T3	PASS
NOT	NOT 0101	1010	tb_alu, tb_cpu T3	PASS
LDI	LDI R3, 1010	R3 = 1010	tb_cpu T1, T3	PASS
MOV	R1 = 0110; MOV R2,R1	R2 = 0110	tb_cpu T3	PASS
Zero flag	Result 0000	Zero = 1	tb_alu, tb_cpu T2, T3	PASS
PC increment	Reset, run in order	PC 0 → 1	tb_cpu T4	PASS
Jump / branch	JMP / taken JZ to address 4	PC = 0100	tb_cpu T4	PASS
Reset	Assert reset	PC, registers, flags, pipeline at initial values	tb_cpu T5	PASS

Table 32. *Every test in the project plan, with its result*

13.2 Module-level testbenches

tb_alu.v

```
`timescale 1ns / 1ps
//
// CONNECTIONS
//   Calls       : alu.v (module alu)
//   Called by   : nobody - this is a testbench (top of simulation)
// ============================================================================
// FILE: tb_alu.v
// MODULE-LEVEL TEST 1: the ALU on its own (no clock needed).
// Covers every ALU operation in the project plan, plus the Zero flag
// and the Carry / no-borrow convention.
// ============================================================================
module tb_alu;
    reg  [3:0] a, b;
    reg  [2:0] alu_ctrl;
    wire [3:0] result;
    wire       carry, zero;
    integer errors = 0;
    integer tests  = 0;
    // CALLS: module alu from file alu.v
    alu DUT (.a(a), .b(b), .alu_ctrl(alu_ctrl),
             .result(result), .carry(carry), .zero(zero));
    // apply one test and compare with the expected values
    task test;
        input [8*12:1] name;
        input [3:0]    in_a, in_b;
        input [2:0]    op;
        input [3:0]    exp_result;
        input          exp_carry;
        input          exp_zero;
        begin
            a = in_a;  b = in_b;  alu_ctrl = op;
            #10;
            tests = tests + 1;
            if (result == exp_result && carry == exp_carry && zero == exp_zero)
                $display("PASS  %0s a=%b b=%b -> result=%b carry=%b zero=%b",
                         name, a, b, result, carry, zero);
            else begin
                $display("FAIL  %0s a=%b b=%b -> result=%b carry=%b zero=%b",
                         name, a, b, result, carry, zero);
                $display("      expected result=%b carry=%b zero=%b",
                         exp_result, exp_carry, exp_zero);
                errors = errors + 1;
            end
        end
    endtask
    initial begin
        // The task 'test' used below is defined above in this same file.
        $display("\n===== tb_alu: ALU module test =====");
        //      name            a        b        ctrl  result   C  Z
        test("ADD 5+3     ", 4'b0101, 4'b0011, 3'd0, 4'b1000, 0, 0);
        test("ADD 15+1    ", 4'b1111, 4'b0001, 3'd0, 4'b0000, 1, 1);
        test("SUB 5-3     ", 4'b0101, 4'b0011, 3'd1, 4'b0010, 1, 0);
        test("SUB 3-5     ", 4'b0011, 4'b0101, 3'd1, 4'b1110, 0, 0);
        test("SUB 7-7     ", 4'b0111, 4'b0111, 3'd1, 4'b0000, 1, 1);
        test("AND         ", 4'b1100, 4'b1010, 3'd2, 4'b1000, 0, 0);
        test("OR          ", 4'b1100, 4'b1010, 3'd3, 4'b1110, 0, 0);
        test("XOR         ", 4'b1100, 4'b1010, 3'd4, 4'b0110, 0, 0);
        test("XOR to zero ", 4'b0110, 4'b0110, 3'd4, 4'b0000, 0, 1);
        test("NOT 0101    ", 4'b0101, 4'b0000, 3'd5, 4'b1010, 0, 0);
        test("PASS b      ", 4'b0000, 4'b1001, 3'd6, 4'b1001, 0, 0);
        if (errors == 0)
            $display("tb_alu PASSED: %0d of %0d tests OK\n", tests, tests);
        else
            $display("tb_alu FAILED: %0d of %0d tests failed\n", errors, tests);
        $finish;
    end
endmodule
```

tb_register_file.v

```
`timescale 1ns / 1ps
//
// CONNECTIONS
//   Calls       : register_file.v (module register_file)
//   Called by   : nobody - this is a testbench (top of simulation)
// ============================================================================
// FILE: tb_register_file.v
// MODULE-LEVEL TEST 2: the register file on its own.
// Checks: reset clears registers, each register can be written and read,
// both read ports work at the same time, and we = 0 blocks writing.
// ============================================================================
module tb_register_file;
    reg        clk = 0;
    reg        rst = 1;
    reg  [1:0] ra = 0, rb = 0, wa = 0;
    reg  [3:0] wd = 0;
    reg        we = 0;
    wire [3:0] qa, qb;
    integer errors = 0;
    integer tests  = 0;
    always #5 clk = ~clk;
    // CALLS: module register_file from file register_file.v
    register_file DUT (.clk(clk), .rst(rst), .ra(ra), .rb(rb),
                       .wa(wa), .wd(wd), .we(we), .qa(qa), .qb(qb));
    // compare one 4-bit value
    task check;
        input [8*26:1] name;
        input [3:0]    actual;
        input [3:0]    expected;
        begin
            tests = tests + 1;
            if (actual == expected)
                $display("PASS  %0s = %b", name, actual);
            else begin
                $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
                errors = errors + 1;
            end
        end
    endtask
    // write one register (takes one clock edge)
    task write_reg;
        input [1:0] addr;
        input [3:0] data;
        begin
            wa = addr;  wd = data;  we = 1;
            @(posedge clk);  #1;
            we = 0;
        end
    endtask
    initial begin
        // The tasks 'check' and 'write_reg' used below are defined above
        // in this same file.
        $display("\n===== tb_register_file: register file test =====");
        // 1. reset
        @(posedge clk); #1;
        rst = 0;
        ra = 0; rb = 3; #1;
        check("after reset R0 (port A)", qa, 4'b0000);
        check("after reset R3 (port B)", qb, 4'b0000);
        // 2. write a different value into each register
        write_reg(2'd0, 4'b0001);
        write_reg(2'd1, 4'b0101);
        write_reg(2'd2, 4'b1010);
        write_reg(2'd3, 4'b1111);
        // 3. read them back on both ports
        ra = 0; rb = 1; #1;
        check("R0 on port A", qa, 4'b0001);
        check("R1 on port B", qb, 4'b0101);
        ra = 2; rb = 3; #1;
        check("R2 on port A", qa, 4'b1010);
        check("R3 on port B", qb, 4'b1111);
        ra = 1; rb = 1; #1;
```

tb_register_file.v

```
        check("R1 on both ports (A)", qa, 4'b0101);
        check("R1 on both ports (B)", qb, 4'b0101);
        // 4. we = 0 must NOT change a register
        wa = 2; wd = 4'b0000; we = 0;
        @(posedge clk); #1;
        ra = 2; #1;
        check("R2 unchanged when we=0", qa, 4'b1010);
        // 5. reset again clears everything
        rst = 1; @(posedge clk); #1; rst = 0;
        ra = 3; rb = 2; #1;
        check("R3 after second reset", qa, 4'b0000);
        check("R2 after second reset", qb, 4'b0000);
        if (errors == 0)
            $display("tb_register_file PASSED: %0d of %0d checks OK\n", tests, tests);
        else
            $display("tb_register_file FAILED: %0d of %0d checks failed\n", errors, tests);
        $finish;
    end
endmodule
```

tb_control_unit packs all decoder outputs into one 11-bit word and compares it with the expected row of the decoder truth table for every opcode. It then checks the branch unit in all six situations of its truth table.

tb_control_unit.v

```
`timescale 1ns / 1ps
//
// CONNECTIONS
//   Calls       : control_unit.v (module control_unit)
//   Called by   : nobody - this is a testbench (top of simulation)
// ========================================================================
// FILE: tb_control_unit.v
// MODULE-LEVEL TEST 3: the control unit (no clock needed).
// Part A: decoder output for EVERY opcode, compared with the
//         control-signal truth table in the report.
// Part B: branch unit - when is a jump taken and the pipeline flushed?
// ========================================================================
module tb_control_unit;
    reg  [3:0] opcode    = 4'b0000;
    reg        ex_is_jmp = 0;
    reg        ex_is_jz  = 0;
    reg        zero_flag = 0;
    wire [2:0] alu_ctrl;
    wire       alu_src_sel, wb_sel, reg_we, mem_we;
    wire       zero_we, carry_we, is_jmp, is_jz;
    wire       pc_sel, flush;
    integer errors = 0;
    integer tests  = 0;
    // CALLS: module control_unit from file control_unit.v
    control_unit DUT (
        .opcode(opcode), .alu_ctrl(alu_ctrl), .alu_src_sel(alu_src_sel),
        .wb_sel(wb_sel), .reg_we(reg_we), .mem_we(mem_we),
        .zero_we(zero_we), .carry_we(carry_we), .is_jmp(is_jmp), .is_jz(is_jz),
        .ex_is_jmp(ex_is_jmp), .ex_is_jz(ex_is_jz), .zero_flag(zero_flag),
        .pc_sel(pc_sel), .flush(flush)
    );
    // All decoder outputs packed into one 11-bit word, in this order:
    // alu_ctrl[2:0] alu_src_sel wb_sel reg_we mem_we zero_we carry_we is_jmp is_jz
    wire [10:0] signals = {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
                           zero_we, carry_we, is_jmp, is_jz};
    // check the decoder output for one opcode
    task check_op;
        input [8*6:1] name;
        input [3:0]   op;
        input [10:0]  expected;
        begin
            opcode = op;  #1;
            tests = tests + 1;
            if (signals == expected)
```

tb_control_unit.v

```
                $display("PASS  %0s alu=%0d src=%b wb=%b rwe=%b mwe=%b zwe=%b cwe=%b jmp=%b jz=%b",
                         name, alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
                         zero_we, carry_we, is_jmp, is_jz);
            else begin
                $display("FAIL  %0s got %b, expected %b", name, signals, expected);
                errors = errors + 1;
            end
        end
    endtask
    // check the branch unit for one situation
    task check_branch;
        input [8*24:1] name;
        input          jmp, jz, z;
        input          exp_pc_sel;
        begin
            ex_is_jmp = jmp;  ex_is_jz = jz;  zero_flag = z;  #1;
            tests = tests + 1;
            if (pc_sel == exp_pc_sel && flush == exp_pc_sel)
                $display("PASS  %0s pc_sel=%b flush=%b", name, pc_sel, flush);
            else begin
                $display("FAIL  %0s pc_sel=%b flush=%b (expected %b)",
                         name, pc_sel, flush, exp_pc_sel);
                errors = errors + 1;
            end
        end
    endtask
    initial begin
        // The tasks 'check_op' and 'check_branch' used below are defined
        // above in this same file.
        $display("\n===== tb_control_unit: control unit test =====");
        $display("--- Part A: decoder (DECODE stage) ---");
        //          name     opcode    alu   src   wb   rwe  mwe  zwe  cwe  jmp  jz
        check_op("ADD   ", 4'b0000, {3'd0, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b0});
        check_op("SUB   ", 4'b0001, {3'd1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b0});
        check_op("AND   ", 4'b0010, {3'd2, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
        check_op("OR    ", 4'b0011, {3'd3, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
        check_op("XOR   ", 4'b0100, {3'd4, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
        check_op("NOT   ", 4'b0101, {3'd5, 1'b0, 1'b0, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0});
        check_op("LDI   ", 4'b0110, {3'd6, 1'b1, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("MOV   ", 4'b0111, {3'd6, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("LOAD  ", 4'b1000, {3'd6, 1'b1, 1'b1, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("STORE ", 4'b1001, {3'd6, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("JMP   ", 4'b1010, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b1, 1'b0});
        check_op("JZ    ", 4'b1011, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b1});
        check_op("RES-C ", 4'b1100, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("RES-D ", 4'b1101, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("RES-E ", 4'b1110, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        check_op("RES-F ", 4'b1111, {3'd0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0});
        $display("--- Part B: branch unit (EXECUTE stage) ---");
        //              name                      jmp jz  Z   pc_sel
        check_branch("no jump instruction     ", 0, 0, 0, 0);
        check_branch("no jump, Zero = 1       ", 0, 0, 1, 0);
        check_branch("JMP                     ", 1, 0, 0, 1);
        check_branch("JMP (Zero = 1)          ", 1, 0, 1, 1);
        check_branch("JZ with Zero = 0        ", 0, 1, 0, 0);
        check_branch("JZ with Zero = 1        ", 0, 1, 1, 1);
        if (errors == 0)
            $display("tb_control_unit PASSED: %0d of %0d checks OK\n", tests, tests);
        else
            $display("tb_control_unit FAILED: %0d of %0d checks failed\n", errors, tests);
        $finish;
    end
endmodule
```

13.3 CPU-level testbench

- **load_program** fills the instruction memory with NOP, clears the data memory and loads a .mem file.
- **reset_cpu** holds reset for 2 clocks.
- **finish(addr)** waits until the instruction at that address is in EX and then lets the clock edge write its result. Because of the pipeline, "waiting N cycles" is not reliable after jumps, so tests wait for an instruction instead.
- **check / check_bit** compare a value and print PASS or FAIL.
- Test 1 prints **what each pipeline stage holds in every clock cycle**; the other tests print one line per executed instruction. A cycle counter shows how many clocks a program took.
- `CPU.RF.r1` is a hierarchical reference (inside CPU, inside RF, the signal r1); it works in simulation only.
- The X check XORs all key bits together: the answer is X only if some bit is X.

tb_cpu.v

```
`timescale 1ns / 1ps
//
// CONNECTIONS
//   Calls       : cpu_top.v (module cpu_top), which uses all other design files;
//                 programs program.mem, prog_arith.mem, prog_logic.mem, prog_jump.mem
//   Called by   : nobody - this is a testbench (top of simulation)
// ============================================================================
// FILE: tb_cpu.v
// CPU-LEVEL TEST: runs programs on the complete pipelined CPU and checks
// the result after each important instruction (project plan, Section 8).
//
//   TEST 1  program.mem    : demonstration program (plan Appendix A)
//                            + a cycle-by-cycle view of the pipeline
//   TEST 2  prog_arith.mem : ADD, SUB, Carry, STORE, LOAD, Zero
//   TEST 3  prog_logic.mem : AND, OR, XOR, MOV, NOT, LDI, Zero
//   TEST 4  prog_jump.mem  : PC increment, JMP to 0100, JZ taken/not taken,
//                            flushing of wrongly fetched instructions
//   TEST 5  reset          : reset values and no unknown (X) values
// ============================================================================
module tb_cpu;
    reg clk = 0;
    reg rst = 1;
    integer errors = 0;
    integer checks = 0;
    integer cycle  = 0;            // clock cycles since reset was released
    integer i;
    reg     show_pipeline = 0;     // 1 = print every clock cycle
    always #5 clk = ~clk;          // 10 ns clock period
    wire [3:0]  pc_out, alu_out, wb_data_out;
    wire [11:0] instr_out;
    wire        zero_out, carry_out;
    // CALLS: module cpu_top from file cpu_top.v
    cpu_top #(.PROGRAM_FILE("program.mem")) CPU (
        .clk(clk), .rst(rst),
        .pc_out(pc_out), .instr_out(instr_out), .alu_out(alu_out),
        .wb_data_out(wb_data_out), .zero_out(zero_out), .carry_out(carry_out)
    );
    // count clock cycles
    always @(posedge clk) begin
        if (rst) cycle <= 0;
        else     cycle <= cycle + 1;
    end
    // ------------------------------------------------------------------
    // Trace. Printed in the middle of each clock cycle.
    //   show_pipeline = 1 : what each stage holds, every cycle
    //   otherwise         : one line when an instruction is in EXECUTE
    // CPU.RF.r1 means: inside CPU (cpu_top.v), inside RF (register_file.v),
    // the signal r1. CPU.id_instr / CPU.id_valid come from
```

tb_cpu.v

```
    // instruction_register.v (instance IR); CPU.ex_instr / CPU.ex_valid
    // come from pipeline_register.v (instance ID_EX).
    // ------------------------------------------------------------------
    always @(negedge clk) begin
        if (!rst && show_pipeline) begin
            $write("   cycle %2d | IF: addr %2d | ID: ", cycle + 1, CPU.pc);
            if (CPU.id_valid) $write("%h (addr %2d)", CPU.id_instr, CPU.id_pc);
            else              $write("bubble        ");
            $write(" | EX: ");
            if (CPU.ex_valid) $write("%h (addr %2d)", CPU.ex_instr, CPU.ex_pc);
            else              $write("bubble        ");
            $display(" | R1=%b R2=%b R3=%b", CPU.RF.r1, CPU.RF.r2, CPU.RF.r3);
        end
        else if (!rst && CPU.ex_valid)
            $display("   EX addr %2d: %h | R0=%b R1=%b R2=%b R3=%b | Z=%b C=%b",
                     CPU.ex_pc, CPU.ex_instr, CPU.RF.r0, CPU.RF.r1, CPU.RF.r2, CPU.RF.r3,
                     zero_out, carry_out);
    end
    // ==================================================================
    // Helper tasks
    // ==================================================================
    // reset the CPU for 2 clocks
    task reset_cpu;
        begin
            rst = 1;
            @(posedge clk); @(posedge clk); #1;
            rst = 0;
        end
    endtask
    // NOTE: CPU.IMEM.mem is the array 'mem' inside instruction_memory.v
    //       (instance IMEM in cpu_top.v); CPU.DMEM.mem is the array 'mem'
    //       inside data_memory.v (instance DMEM in cpu_top.v).
    // load a program file and clear the data memory
    task load_program;
        input [8*20:1] file_name;
        begin
            rst = 1;
            for (i = 0; i < 16; i = i + 1) begin
                CPU.IMEM.mem[i] = 12'hC00;
                CPU.DMEM.mem[i] = 4'b0000;
            end
            $readmemh(file_name, CPU.IMEM.mem);
        end
    endtask
    // NOTE: CPU.ex_valid and CPU.ex_pc are outputs of pipeline_register.v
    //       (instance ID_EX in cpu_top.v).
    // wait until the instruction at address 'addr' has finished
    // (it is in EXECUTE, and the next clock edge writes its result)
    task finish;
        input [3:0] addr;
        begin
            @(negedge clk);
            while (!(CPU.ex_valid && CPU.ex_pc == addr))
                @(negedge clk);
            @(posedge clk);
            #1;
        end
    endtask
    // compare a 4-bit value
    task check;
        input [8*44:1] name;
        input [3:0]    actual;
        input [3:0]    expected;
        begin
            checks = checks + 1;
            if (actual === expected)
                $display("PASS  %0s = %b", name, actual);
            else begin
                $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
                errors = errors + 1;
            end
        end
    endtask
```

tb_cpu.v

```
    // compare a 1-bit value
    task check_bit;
        input [8*44:1] name;
        input          actual;
        input          expected;
        begin
            checks = checks + 1;
            if (actual === expected)
                $display("PASS  %0s = %b", name, actual);
            else begin
                $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
                errors = errors + 1;
            end
        end
    endtask
    // ============================================================
    // Tests
    // ============================================================
    initial begin
        // The tasks used below (reset_cpu, load_program, finish, check,
        // check_bit) are defined above in this same file.
        // ---------------- TEST 1: demonstration program ----------------
        $display("\n===== TEST 1: demonstration program (program.mem) =====");
        show_pipeline = 1;
        reset_cpu;
        finish(2);
        check("ADD R1,R2  -> R1 (5+3)", CPU.RF.r1, 4'b1000);
        finish(4);
        check("LDI R3,1010 -> R3", CPU.RF.r3, 4'b1010);
        check("XOR R1,R3  -> R1", CPU.RF.r1, 4'b0010);
        check("R2 still 0011", CPU.RF.r2, 4'b0011);
        $display("      5 instructions finished after %0d clock cycles", cycle);
        finish(5);
        check("JMP 0101: PC (fetch address) = 5", pc_out, 4'd5);
        finish(5);
        show_pipeline = 0;
        check("JMP runs again: program loops at 5", pc_out, 4'd5);
        // ---------------- TEST 2: arithmetic and memory ----------------
        $display("\n===== TEST 2: arithmetic and memory (prog_arith.mem) =====");
        load_program("prog_arith.mem");
        reset_cpu;
        finish(2);
        check    ("ADD R1,R2 (0101+0011) -> R1", CPU.RF.r1, 4'b1000);
        check_bit("ADD: Carry", carry_out, 1'b0);
        check_bit("ADD: Zero", zero_out, 1'b0);
        finish(3);
        check    ("STORE R1,0 -> data_memory[0]", CPU.DMEM.mem[0], 4'b1000);
        finish(5);
        check    ("SUB R1,R2 (0101-0011) -> R1", CPU.RF.r1, 4'b0010);
        check_bit("SUB: Zero", zero_out, 1'b0);
        check_bit("SUB: Carry (1 = no borrow)", carry_out, 1'b1);
        finish(6);
        check    ("STORE R1,1 -> data_memory[1]", CPU.DMEM.mem[1], 4'b0010);
        finish(7);
        check    ("LOAD R0,0 -> R0", CPU.RF.r0, 4'b1000);
        finish(8);
        check    ("SUB R2,R2 -> R2", CPU.RF.r2, 4'b0000);
        check_bit("Zero flag after 0000 result", zero_out, 1'b1);
        $display("      9 instructions finished after %0d clock cycles (pipeline: 9 + 2)", cycle);
        // ---------------- TEST 3: logic, MOV, NOT, LDI ----------------
        $display("\n===== TEST 3: logic operations (prog_logic.mem) =====");
        load_program("prog_logic.mem");
        reset_cpu;
        finish(3);
        check    ("AND 1100,1010 -> R3", CPU.RF.r3, 4'b1000);
        finish(5);
        check    ("OR  1100,1010 -> R3", CPU.RF.r3, 4'b1110);
        finish(6);
        check    ("XOR 1100,1010 -> R1", CPU.RF.r1, 4'b0110);
        finish(7);
        check    ("MOV R2,R1 (R1=0110) -> R2", CPU.RF.r2, 4'b0110);
        finish(9);
        check    ("NOT 0101 -> R0", CPU.RF.r0, 4'b1010);
```

tb_cpu.v

```
        finish(10);
        check    ("LDI R3,1010 -> R3", CPU.RF.r3, 4'b1010);
        finish(11);
        check    ("XOR R3,R3 -> R3", CPU.RF.r3, 4'b0000);
        check_bit("Zero flag after 0000 result", zero_out, 1'b1);
        $display("      12 instructions finished after %0d clock cycles (pipeline: 12 + 2)", cycle);
        // ---------------- TEST 4: PC increment and jumps ----------------
        $display("\n===== TEST 4: PC and jumps (prog_jump.mem) =====");
        load_program("prog_jump.mem");
        rst = 1;
        @(posedge clk); @(posedge clk); #1;
        check    ("PC after reset", pc_out, 4'd0);
        rst = 0;
        @(posedge clk); #1;
        check    ("PC increment (0 -> 1)", pc_out, 4'd1);
        finish(1);
        check    ("JMP 4: PC becomes 0100", pc_out, 4'b0100);
        check_bit("JMP flushed the ID/EX register", CPU.ex_valid, 1'b0);
        finish(5);
        check_bit("SUB R1,R1 sets Zero", zero_out, 1'b1);
        finish(6);
        check    ("JZ 9 taken: PC = 9", pc_out, 4'd9);
        finish(10);
        check_bit("ADD R3,R3 clears Zero", zero_out, 1'b0);
        finish(11);
        check    ("JZ 14 not taken: next in EX is addr 12", CPU.ex_pc, 4'd12);
        finish(13);
        check    ("R0 = 1 (JMP skipped addresses 2, 3)", CPU.RF.r0, 4'd1);
        check    ("R2 = 6 (JZ skipped 7, 8; 14 never ran)", CPU.RF.r2, 4'd6);
        check    ("Program stops at PC = 13", pc_out, 4'd13);
        // ---------------- TEST 5: reset and unknown values ----------------
        $display("\n===== TEST 5: reset =====");
        rst = 1;
        @(posedge clk); #1;
        check    ("PC after reset", pc_out, 4'd0);
        check    ("R0 after reset", CPU.RF.r0, 4'd0);
        check    ("R2 after reset", CPU.RF.r2, 4'd0);
        check_bit("Zero after reset", zero_out, 1'b0);
        check_bit("Carry after reset", carry_out, 1'b0);
        check_bit("IF/ID empty after reset", CPU.id_valid, 1'b0);
        check_bit("ID/EX empty after reset", CPU.ex_valid, 1'b0);
        rst = 0;
        repeat (6) @(posedge clk);
        #1;
        // ^{...} XORs all bits together: the answer is X if ANY bit is X
        checks = checks + 1;
        if ((^{pc_out, instr_out, alu_out, wb_data_out, zero_out, carry_out,
               CPU.RF.r0, CPU.RF.r1, CPU.RF.r2, CPU.RF.r3}) === 1'bx) begin
            $display("FAIL  unknown (X) value found on key outputs");
            errors = errors + 1;
        end
        else
            $display("PASS  no unknown (X) values on key outputs or registers");
        // ---------------- summary ----------------
        $display("\n================================================");
        if (errors == 0)
            $display("tb_cpu PASSED: %0d of %0d checks OK", checks, checks);
        else
            $display("tb_cpu FAILED: %0d of %0d checks failed", errors, checks);
        $display("================================================\n");
        $finish;
    end
endmodule
```

14. Simulation Results

14.1 Module-level results

tb_alu output

```
===== tb_alu: ALU module test =====
PASS  ADD 5+3      a=0101 b=0011 -> result=1000 carry=0 zero=0
PASS  ADD 15+1     a=1111 b=0001 -> result=0000 carry=1 zero=1
PASS  SUB 5-3      a=0101 b=0011 -> result=0010 carry=1 zero=0
PASS  SUB 3-5      a=0011 b=0101 -> result=1110 carry=0 zero=0
PASS  SUB 7-7      a=0111 b=0111 -> result=0000 carry=1 zero=1
PASS  AND          a=1100 b=1010 -> result=1000 carry=0 zero=0
PASS  OR           a=1100 b=1010 -> result=1110 carry=0 zero=0
PASS  XOR          a=1100 b=1010 -> result=0110 carry=0 zero=0
PASS  XOR to zero  a=0110 b=0110 -> result=0000 carry=0 zero=1
PASS  NOT 0101     a=0101 b=0000 -> result=1010 carry=0 zero=0
PASS  PASS b       a=0000 b=1001 -> result=1001 carry=0 zero=0
tb_alu PASSED: 11 of 11 tests OK
```

tb_register_file output

```
===== tb_register_file: register file test =====
PASS  after reset R0 (port A) = 0000
PASS  after reset R3 (port B) = 0000
PASS  R0 on port A = 0001
PASS  R1 on port B = 0101
PASS  R2 on port A = 1010
PASS  R3 on port B = 1111
PASS  R1 on both ports (A) = 0101
PASS  R1 on both ports (B) = 0101
PASS  R2 unchanged when we=0 = 1010
PASS  R3 after second reset = 0000
PASS  R2 after second reset = 0000
tb_register_file PASSED: 11 of 11 checks OK
```

tb_control_unit output

```
===== tb_control_unit: control unit test =====
--- Part A: decoder (DECODE stage) ---
PASS  ADD    alu=0 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=1 jmp=0 jz=0
PASS  SUB    alu=1 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=1 jmp=0 jz=0
PASS  AND    alu=2 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=0 jmp=0 jz=0
PASS  OR     alu=3 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=0 jmp=0 jz=0
PASS  XOR    alu=4 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=0 jmp=0 jz=0
PASS  NOT    alu=5 src=0 wb=0 rwe=1 mwe=0 zwe=1 cwe=0 jmp=0 jz=0
PASS  LDI    alu=6 src=1 wb=0 rwe=1 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  MOV    alu=6 src=0 wb=0 rwe=1 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  LOAD   alu=6 src=1 wb=1 rwe=1 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  STORE  alu=6 src=1 wb=0 rwe=0 mwe=1 zwe=0 cwe=0 jmp=0 jz=0
PASS  JMP    alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=1 jz=0
PASS  JZ     alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=0 jz=1
PASS  RES-C  alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  RES-D  alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  RES-E  alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
PASS  RES-F  alu=0 src=0 wb=0 rwe=0 mwe=0 zwe=0 cwe=0 jmp=0 jz=0
--- Part B: branch unit (EXECUTE stage) ---
PASS  no jump instruction      pc_sel=0 flush=0
PASS  no jump, Zero = 1        pc_sel=0 flush=0
PASS  JMP                      pc_sel=1 flush=1
PASS  JMP (Zero = 1)           pc_sel=1 flush=1
PASS  JZ with Zero = 0         pc_sel=0 flush=0
PASS  JZ with Zero = 1         pc_sel=1 flush=1
tb_control_unit PASSED: 22 of 22 checks OK
```

14.2 CPU-level results

In the Test 1 trace, each line shows one clock cycle: the address in IF, the instruction in ID, the instruction in EX, and the register values during that cycle (a result appears in the next line, after the clock edge).

tb_cpu output

```
===== TEST 1: demonstration program (program.mem) =====
   cycle  1 | IF: addr  0 | ID: bubble         | EX: bubble         | R1=0000 R2=0000 R3=0000
   cycle  2 | IF: addr  1 | ID: 645 (addr  0) | EX: bubble         | R1=0000 R2=0000 R3=0000
   cycle  3 | IF: addr  2 | ID: 683 (addr  1) | EX: 645 (addr  0) | R1=0000 R2=0000 R3=0000
   cycle  4 | IF: addr  3 | ID: 060 (addr  2) | EX: 683 (addr  1) | R1=0101 R2=0000 R3=0000
   cycle  5 | IF: addr  4 | ID: 6ca (addr  3) | EX: 060 (addr  2) | R1=0101 R2=0011 R3=0000
PASS  ADD R1,R2  -> R1 (5+3) = 1000
   cycle  6 | IF: addr  5 | ID: 470 (addr  4) | EX: 6ca (addr  3) | R1=1000 R2=0011 R3=0000
   cycle  7 | IF: addr  6 | ID: a05 (addr  5) | EX: 470 (addr  4) | R1=1000 R2=0011 R3=1010
PASS  LDI R3,1010 -> R3 = 1010
PASS  XOR R1,R3  -> R1 = 0010
PASS  R2 still 0011 = 0011
      5 instructions finished after 7 clock cycles
   cycle  8 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS  JMP 0101: PC (fetch address) = 5 = 0101
   cycle  9 | IF: addr  5 | ID: bubble         | EX: bubble         | R1=0010 R2=0011 R3=1010
   cycle 10 | IF: addr  6 | ID: a05 (addr  5) | EX: bubble         | R1=0010 R2=0011 R3=1010
   cycle 11 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS  JMP runs again: program loops at 5 = 0101
===== TEST 2: arithmetic and memory (prog_arith.mem) =====
   EX addr  0: 645 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: 683 | R0=0000 R1=0101 R2=0000 R3=0000 | Z=0 C=0
   EX addr  2: 060 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS  ADD R1,R2 (0101+0011) -> R1 = 1000
PASS  ADD: Carry = 0
PASS  ADD: Zero = 0
   EX addr  3: 940 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
PASS  STORE R1,0 -> data_memory[0] = 1000
   EX addr  4: 645 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
   EX addr  5: 160 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS  SUB R1,R2 (0101-0011) -> R1 = 0010
PASS  SUB: Zero = 0
PASS  SUB: Carry (1 = no borrow) = 1
   EX addr  6: 941 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  STORE R1,1 -> data_memory[1] = 0010
   EX addr  7: 800 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  LOAD R0,0 -> R0 = 1000
   EX addr  8: 1a0 | R0=1000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  SUB R2,R2 -> R2 = 0000
PASS  Zero flag after 0000 result = 1
      9 instructions finished after 11 clock cycles (pipeline: 9 + 2)
===== TEST 3: logic operations (prog_logic.mem) =====
   EX addr  0: 64c | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: 68a | R0=0000 R1=1100 R2=0000 R3=0000 | Z=0 C=0
   EX addr  2: 7d0 | R0=0000 R1=1100 R2=1010 R3=0000 | Z=0 C=0
   EX addr  3: 2e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS  AND 1100,1010 -> R3 = 1000
   EX addr  4: 7d0 | R0=0000 R1=1100 R2=1010 R3=1000 | Z=0 C=0
   EX addr  5: 3e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS  OR  1100,1010 -> R3 = 1110
   EX addr  6: 460 | R0=0000 R1=1100 R2=1010 R3=1110 | Z=0 C=0
PASS  XOR 1100,1010 -> R1 = 0110
   EX addr  7: 790 | R0=0000 R1=0110 R2=1010 R3=1110 | Z=0 C=0
PASS  MOV R2,R1 (R1=0110) -> R2 = 0110
   EX addr  8: 605 | R0=0000 R1=0110 R2=0110 R3=1110 | Z=0 C=0
   EX addr  9: 500 | R0=0101 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS  NOT 0101 -> R0 = 1010
   EX addr 10: 6ca | R0=1010 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS  LDI R3,1010 -> R3 = 1010
   EX addr 11: 4f0 | R0=1010 R1=0110 R2=0110 R3=1010 | Z=0 C=0
PASS  XOR R3,R3 -> R3 = 0000
PASS  Zero flag after 0000 result = 1
      12 instructions finished after 14 clock cycles (pipeline: 12 + 2)
===== TEST 4: PC and jumps (prog_jump.mem) =====
```

tb_cpu output

```
PASS  PC after reset = 0000
PASS  PC increment (0 -> 1) = 0001
   EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS  JMP 4: PC becomes 0100 = 0100
PASS  JMP flushed the ID/EX register = 0
   EX addr  4: 647 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  5: 150 | R0=0001 R1=0111 R2=0000 R3=0000 | Z=0 C=0
PASS  SUB R1,R1 sets Zero = 1
   EX addr  6: b09 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
PASS  JZ 9 taken: PC = 9 = 1001
   EX addr  9: 6c2 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
   EX addr 10: 0f0 | R0=0001 R1=0000 R2=0000 R3=0010 | Z=1 C=1
PASS  ADD R3,R3 clears Zero = 0
   EX addr 11: b0e | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
PASS  JZ 14 not taken: next in EX is addr 12 = 1100
   EX addr 12: 686 | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
   EX addr 13: a0d | R0=0001 R1=0000 R2=0110 R3=0100 | Z=0 C=0
PASS  R0 = 1 (JMP skipped addresses 2, 3) = 0001
PASS  R2 = 6 (JZ skipped 7, 8; 14 never ran) = 0110
PASS  Program stops at PC = 13 = 1101
===== TEST 5: reset =====
PASS  PC after reset = 0000
PASS  R0 after reset = 0000
PASS  R2 after reset = 0000
PASS  Zero after reset = 0
PASS  Carry after reset = 0
PASS  IF/ID empty after reset = 0
PASS  ID/EX empty after reset = 0
   EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS  no unknown (X) values on key outputs or registers
================================================
tb_cpu PASSED: 44 of 44 checks OK
================================================
```

Result: 88 of 88 checks passed (11 ALU + 11 register file + 22 control unit + 44 CPU), with no unknown (X) values on key outputs after reset.

14.3 Pipeline charts

These charts were made directly from the simulation. Each row is one fetched instruction and each column is a clock cycle; the coloured boxes show which stage the instruction is in.

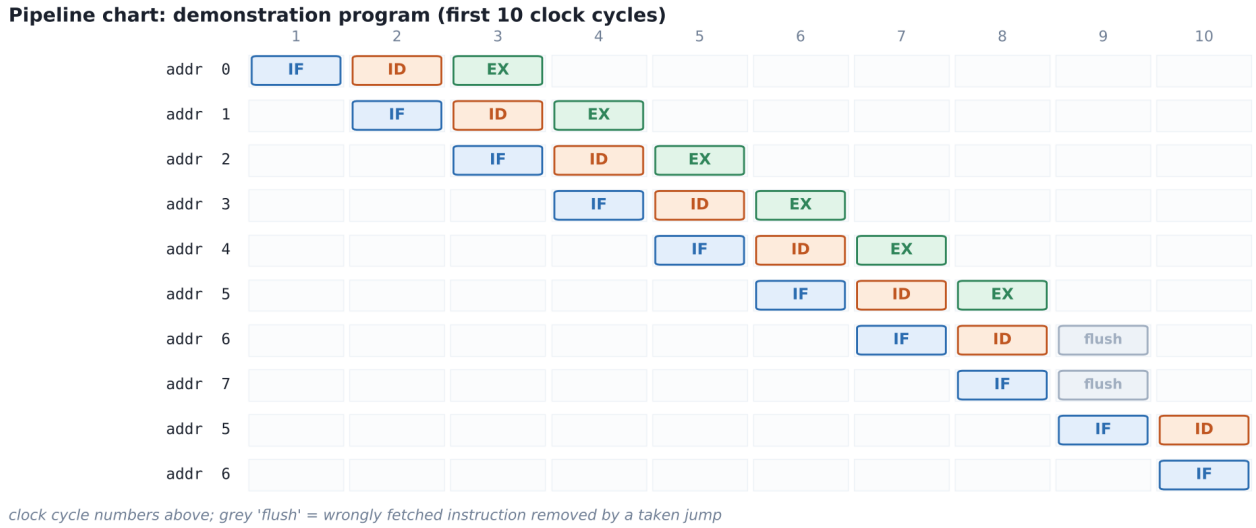


Figure 14. *Pipeline chart of the demonstration program. Instructions 0 to 4 overlap perfectly; JMP at address 5 then flushes the two instructions fetched behind it.*

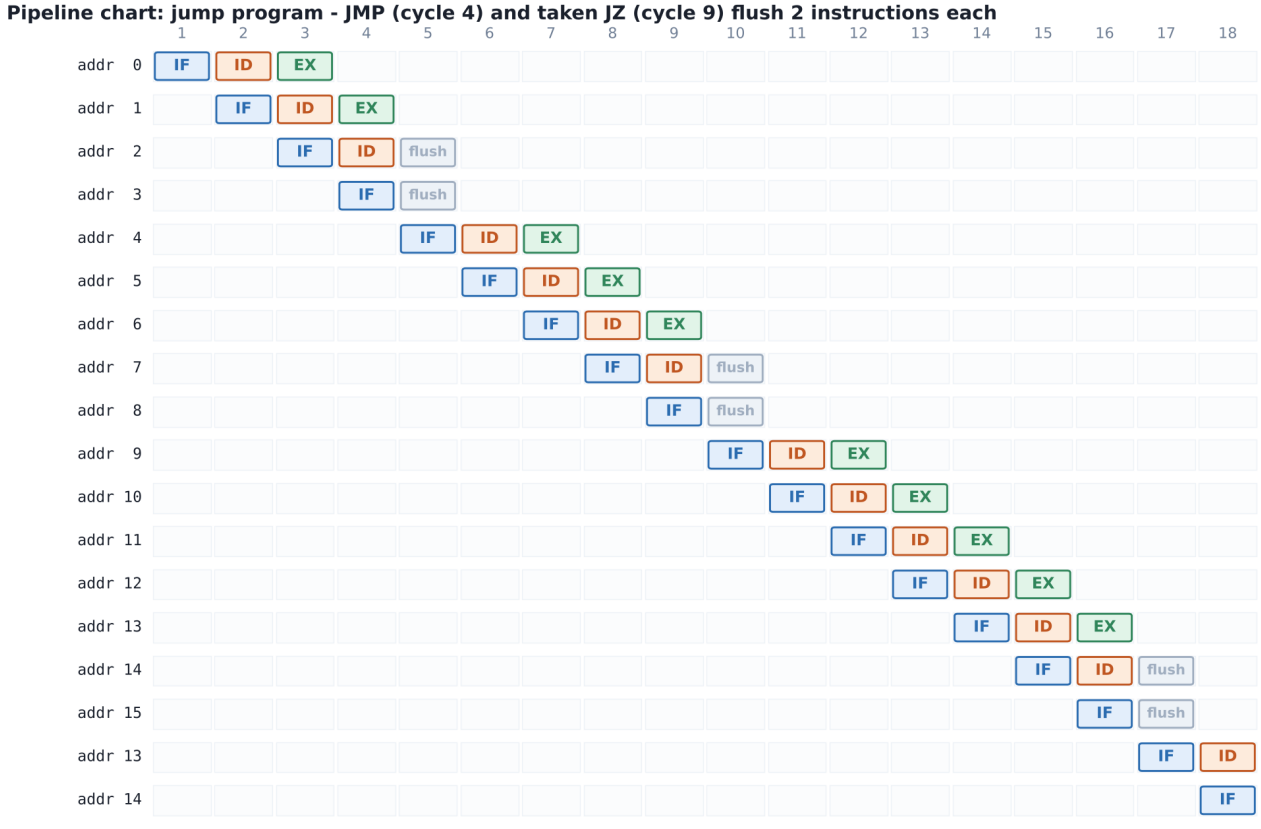


Figure 15. *Pipeline chart of the jump program: JMP 4 and the taken JZ 9 each flush two instructions; the not-taken JZ 14 (address 11) costs nothing.*

14.4 Waveform: demonstration program

This waveform shows what the project plan asks for in the final demonstration: PC, current instructions, ALU result, the value written to the destination register, and the Zero / Carry flags.

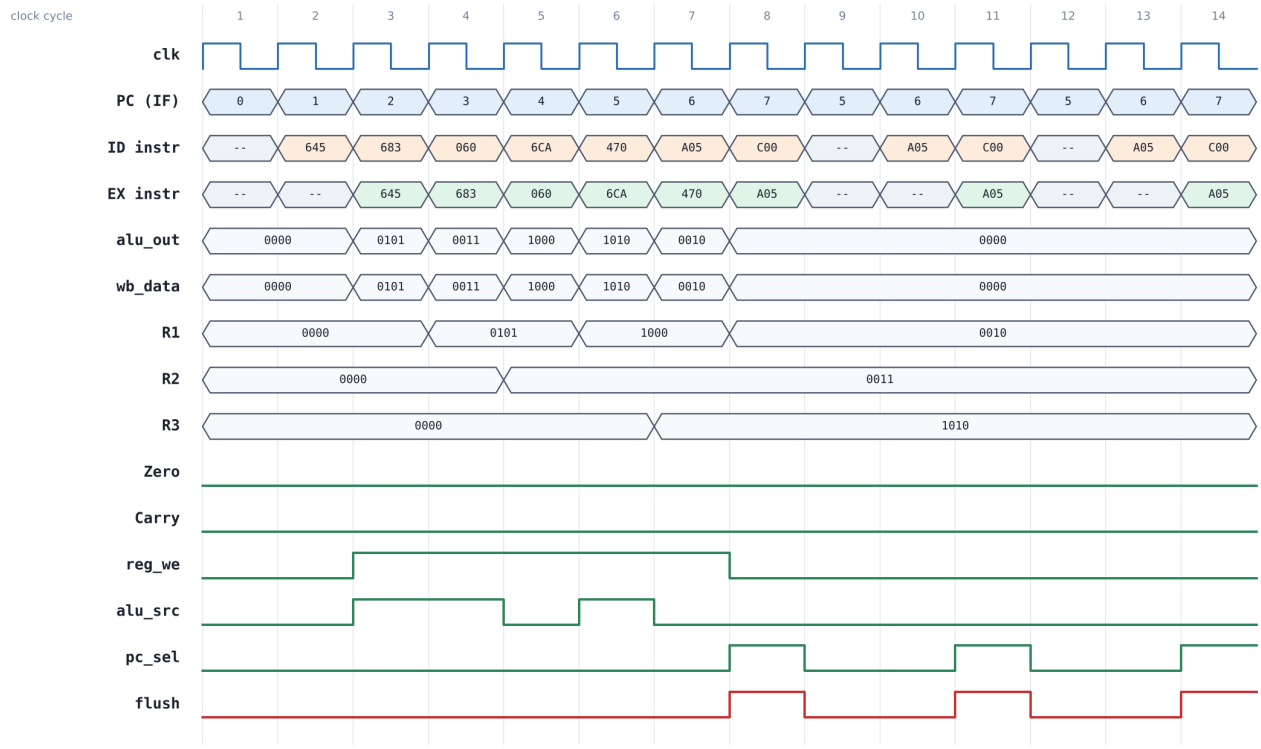


Figure 16. *Simulation waveform of the demonstration program ("--" = empty pipeline slot).*

- **Cycles 1-2:** the pipeline is filling: EX is empty (--) and reg_we = 0.
- **Cycle 3:** LDI R1 (645) is in EX; alu_src = 1 and alu_out = 0101; R1 becomes 0101 at the end of the cycle.
- **Cycles 3-7:** one instruction finishes every clock: R1 = 0101, R2 = 0011, R1 = 1000 (ADD), R3 = 1010, R1 = 0010 (XOR).
- **Cycle 8:** JMP (A05) is in EX; pc_sel = flush = 1; the PC goes back to 5 and the next two slots are empty.
- **From cycle 8:** the loop JMP 5 repeats every 3 cycles; the results stay stable for demonstration.

14.5 Waveform: jumps

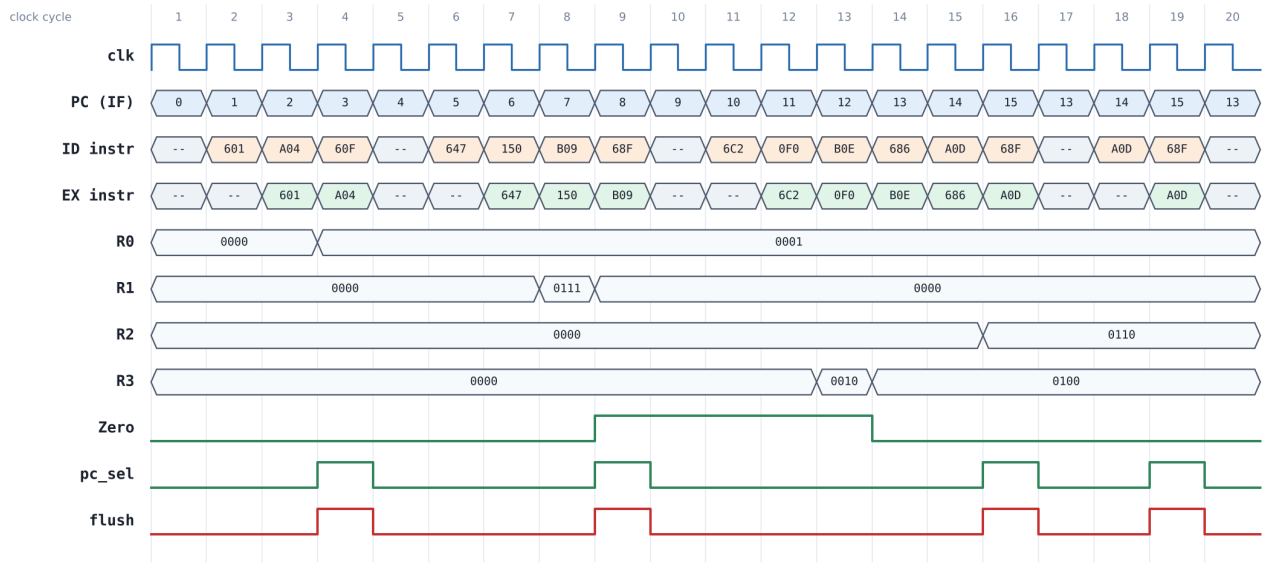


Figure 17. *Simulation waveform of the PC and jump test program.*

- **PC increment:** PC goes 0, 1, 2 ... one step every clock.
- **JMP 4 (cycle 4):** pc_sel = flush = 1; the PC becomes 4 and the ID / EX slots become empty (--).
- **JZ 9 taken (cycle 9):** Zero = 1 after SUB R1,R1, so the PC jumps to 9 and two instructions are flushed.
- **JZ 14 not taken (cycle 14):** Zero = 0, so pc_sel stays 0 and the pipeline continues without any loss.
- **R2 = 0110:** only LDI R2, 6 ever wrote R2; the three LDI R2, 15 instructions were all removed.

14.6 Performance

Program	Instructions run	Old 2-cycle design	3-stage pipeline	Speed-up
Demonstration (to the XOR)	5	10 clocks	7 clocks	1.4x
Arithmetic and memory	9	18 clocks	11 clocks	1.6x
Logic operations	12	24 clocks	14 clocks	1.7x
Long program without jumps	N	2N clocks	N + 2 clocks	close to 2x

Table 33. *Clock cycles needed (pipeline values measured in tb_cpu)*

14.7 Acceptance criteria (project plan Section 8.1)

Criterion	Status	Evidence
All ALU operations give the expected 4-bit results	Met	tb_alu 11/11
Register reads and writes use the correct registers	Met	tb_register_file 11/11
Decoder gives the specified control signals for every opcode	Met	tb_control_unit 22/22
The CPU runs a test program in the intended order	Met	tb_cpu Tests 1-3, traces, pipeline charts
Jump and conditional-branch tests pass	Met	tb_cpu Test 4, jump waveform
No unwanted X values on key outputs after reset	Met	tb_cpu Test 5
Simulation evidence and test results in the report	Met	Chapters 13-14

Table 34. *Acceptance criteria*

15. Running the Project and Demonstration

15.1 Simulation in Vivado

1. Create a new RTL project (no board is needed for simulation).
2. **Add Sources → design sources:** the 11 design files (alu.v ... cpu_top.v). cpu_top becomes the top module.
3. **Add Sources → simulation sources:** tb_alu.v, tb_register_file.v, tb_control_unit.v, tb_cpu.v and all four .mem files (choose "All files" in the file dialog).
4. Right-click a testbench under Simulation Sources and choose **Set as Top**.
5. Run **Simulation → Run Behavioral Simulation**, then type `run all` in the Tcl console.
6. Read the PASS / FAIL lines and the summary. Repeat for each testbench.
7. For the demonstration waveform, set tb_cpu as top and add pc_out, CPU/id_instr, instr_out, alu_out, wb_data_out, zero_out, carry_out and CPU/RF/r1 ... r3 to the waveform window.

15.2 Suggested final demonstration (plan Section 12)

Run the demonstration program. It loads two registers, adds them, does a logical operation and ends in a controlled loop. In the waveform viewer show the PC, the instructions in ID and EX, the ALU result, the value written to the destination register, and the Zero / Carry flags, as in the demonstration waveform above. The pipeline chart is a good slide to explain how the three stages overlap.

15.3 Optional FPGA demonstration

- **Clock:** divide the board clock down to about 1-2 Hz, or use a debounced push-button as a single-step clock.
- **Reset:** connect rst to a push-button.
- **Outputs:** show pc_out, wb_data_out, zero_out and carry_out on LEDs, or alu_out on a seven-segment display.
- **Constraints:** write an .xdc file mapping these ports to the board's pins.

> Testbenches and hierarchical references (such as CPU.RF.r1) are for simulation only; they are not part of the synthesised hardware.

16. Limitations, Extensions and Conclusion

16.1 Limitations and possible extensions

Limitation	Cause	Possible extension
Taken jump costs 2 cycles	Jump decided in EX	Decide JMP in ID (1-cycle penalty) or add branch prediction
16 instructions per program	4-bit PC and jump field	Wider PC with wider jump field or relative jumps
16 data words	4-bit address field	Register-indirect addressing: LOAD Rd, [Rs]
Only one conditional jump	Plan baseline	Use reserved opcodes 1100-1111 for JC, JNZ, shifts or HALT
No signed overflow / negative flags	Not in plan	Add V and N flags
No subroutines	No stack or link register	CALL / RET with a return-address register

Table 35. *Limitations and possible extensions*

16.2 Conclusion

This project designed, implemented and verified a 4-bit RISC-style CPU in Verilog HDL according to the project plan. The 12-bit instruction format and the 12-instruction ISA were fixed first. The datapath was built from a 4-bit ALU, a two-port register file, a program counter, a 16 x 12 instruction memory, a 16 x 4 data memory and three multiplexers, all controlled by a hardwired control unit, and organised as a **3-stage pipeline** with an IF/ID register (the IR) and an ID/EX register.

Every instruction is really fetched, decoded, executed and written back, while the overlap of the stages lets one instruction finish every clock cycle. Data hazards are avoided by reading registers in EX, and control hazards are handled by flushing two instructions after a taken jump. Three module-level testbenches and one CPU-level testbench ran **88 checks, all passing**, covering every test and acceptance criterion in the plan. The waveforms and pipeline charts confirm the overlap of the stages and the correct flushing after jumps.

Appendix: Quick Reference Card

Mnemonic	Opcode	Format (binary)	Example	Hex
ADD Rd, Rs	0000	0000 dd ss 0000	ADD R1, R2	060
SUB Rd, Rs	0001	0001 dd ss 0000	SUB R1, R2	160
AND Rd, Rs	0010	0010 dd ss 0000	AND R3, R2	2E0
OR Rd, Rs	0011	0011 dd ss 0000	OR R3, R2	3E0
XOR Rd, Rs	0100	0100 dd ss 0000	XOR R1, R3	470
NOT Rd	0101	0101 dd 00 0000	NOT R0	500
LDI Rd, imm	0110	0110 dd 00 iiii	LDI R1, 5	645
MOV Rd, Rs	0111	0111 dd ss 0000	MOV R2, R1	790
LOAD Rd, a	1000	1000 dd 00 aaaa	LOAD R0, 0	800
STORE Rd, a	1001	1001 dd 00 aaaa	STORE R1, 1	941
JMP a	1010	1010 00 00 aaaa	JMP 5	A05
JZ a	1011	1011 00 00 aaaa	JZ 9	B09
(reserved)	1100-1111	-	NOP	C00

Table 36. *Instruction encoding quick reference (dd = Rd, ss = Rs)*

Register	Code
R0	00
R1	01
R2	10
R3	11

Table 37. *Register codes*

Pipeline fact	Value
Stages	IF, ID, EX
Latency of one instruction	3 clocks
Throughput (no jumps)	1 instruction per clock
N instructions, no jumps	N + 2 clocks
Penalty of a taken jump	2 clocks (2 instructions flushed)
Penalty of a not-taken JZ	0 clocks

Table 38. *Pipeline summary*